



PROPOSAL PROYEK AKHIR

Rancang Bangun Sistem AI LED Writing Berbasis Web dengan Manajemen Rubrik Penilaian dan Integrasi Generative AI Berbasis Prompt Engineering untuk Generasi Konten dan Skoring Otomatis

M Attar Annaufal Firdaus Nasier
NRP. 3123500050

DOSEN PEMBIMBING

Nur Rosyid Muhtada'i, S.Kom, M.T
NIP. 197403182001121005

Dr. Tita Karlita, S.Kom., M.Kom.
NIP. 197910142002122002

Dr. Selvia Ferdiana Kusuma M.Kom
NIP. 199107222022032005

**PROGRAM STUDI SARJANA TERAPAN
TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026**

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Akreditasi merupakan instrumen penjaminan mutu eksternal yang menilai kualitas penyelenggaraan pendidikan tinggi di Indonesia. Hasil akreditasi berdampak langsung pada daya saing institusi, kepercayaan publik terhadap lulusan, peluang kerja sama dengan industri, hingga akses mahasiswa terhadap beasiswa dan pendanaan riset. Pengelolaan akreditasi di Indonesia dijalankan oleh Badan Akreditasi Nasional Perguruan Tinggi (BAN-PT) bersama sejumlah Lembaga Akreditasi Mandiri (LAM) sesuai rumpun keilmuan, di mana program studi bidang teknik berada di bawah pengelolaan LAM-Teknik.

Di antara seluruh dokumen yang disyaratkan dalam proses akreditasi LAM-Teknik, Laporan Evaluasi Diri (LED) merupakan dokumen paling strategis karena memuat kondisi program studi dari sudut pandang internal. LED disusun mengikuti tujuh kriteria penilaian (C.1 hingga C.7). Setiap kriteria memuat lima sub-bab tetap yaitu latar belakang, kebijakan, indikator kinerja utama, analisis faktor keberhasilan dan penghambat pencapaian, serta strategi perbaikan dan pengembangan menggunakan analisis SWOT. Sub-bab indikator kinerja utama pada setiap kriteria terbagi lagi ke dalam sub-sub-bab yang jumlah dan jenisnya berbeda-beda tergantung nomor kriterianya. Keseluruhan isi LED menggabungkan dua jenis data yang saling terkait, yaitu data kuantitatif dalam bentuk tabel kinerja pada Lembar Kerja Program Studi (LKPS) dan narasi kualitatif analitis yang menafsirkan serta mengevaluasi data tersebut. Kompleksitas struktur ini tercermin pada volume dokumen LED yang dapat mencapai ratusan halaman. Sebagai gambaran, LED program studi STr PJJ Telkom PENS memuat 162 halaman resmi.

Penyusunan LED menuntut keterlibatan tim khusus yang terbagi ke dalam beberapa lapis peran. Koordinator penyusunan bertanggung jawab atas konsistensi dan kelengkapan dokumen secara keseluruhan, sementara penanggungjawab yang ditunjuk per kriteria bertugas menyusun narasi sesuai lingkup tanggung jawabnya, didukung oleh tim pencari data yang mengumpulkan data kuantitatif yang dibutuhkan. Proses penyusunan melalui serangkaian siklus iterasi yang mencakup pemaparan progres, pemberian umpan balik lintas tim, penyempurnaan narasi, dan penyelarasan akhir sebelum dokumen dikompilasi dan dikirimkan. Dalam kasus LED STr PJJ Telkom PENS, proses tersebut memakan waktu tujuh bulan sejak rapat

awal hingga pengiriman ke SAKTI Online, di tengah kewajiban mengajar dan penelitian seluruh anggota tim yang tetap harus berjalan.

Beban penyusunan tersebut semakin berat akibat sejumlah kesulitan teknis. Setiap narasi harus disusun selaras dengan rubrik penilaian LAM-Teknik yang spesifik per kriteria, namun rubrik tersebut tersebar dalam dokumen panduan yang tebal dan tidak praktis untuk dirujuk berulang kali saat menulis. Transformasi data kuantitatif dari tabel LKPS menjadi narasi kualitatif yang analitis rentan menimbulkan ketidaksesuaian ketika data diperbarui di tengah proses penulisan sementara narasi yang sudah ditulis belum ikut disesuaikan. Proses iterasi yang panjang juga menghadirkan risiko hilangnya formulasi narasi yang sebelumnya dianggap lebih baik ketika draf direvisi atau ditulis ulang. Pada praktiknya, tim penyusun bekerja secara terfragmentasi menggunakan kombinasi aplikasi pengolah angka, pengolah kata, dan dokumen panduan yang terpisah, tanpa mekanisme yang menyatukan ketiganya dalam satu alur kerja.

Di era sekarang, kemunculan *Generative AI*, khususnya *Large Language Model* (LLM), menghadirkan peluang nyata untuk meringankan beban penyusunan LED karena kemampuannya menghasilkan narasi panjang secara otomatis dari data terstruktur dalam waktu singkat. Namun, pemanfaatan teknologi ini dalam konteks dokumen formal seperti LED tidak dapat dilakukan begitu saja karena karakteristik LED yang menuntut akurasi data tinggi, kepatuhan terhadap rubrik penilaian yang ketat, dan konsistensi konten lintas ratusan halaman. Kondisi ini menunjukkan bahwa diperlukan sebuah lingkungan kerja terpadu yang dirancang khusus untuk konteks penyusunan LED, di mana *Generative AI* dapat dimanfaatkan sebagai komponen pendukung secara terkendali dan dapat diandalkan.

1.2 PERMASALAHAN

Berdasarkan latar belakang yang telah diuraikan, persoalan praktis dalam penyusunan LED dirumuskan sebagai berikut.

1. Diskrepansi antara data kuantitatif dan narasi kualitatif rentan terjadi sepanjang proses penyusunan LED. Penanggungjawab kriteria umumnya menyusun narasi sambil merujuk data dari tabel LKPS yang masih dalam tahap pengisian oleh tim pencari data. Ketika data pada tabel diperbarui setelah narasi selesai ditulis, misalnya angka rasio dosen berubah akibat koreksi perhitungan di bulan keempat penyusunan, narasi yang sudah jadi tidak ikut diperbarui secara otomatis, sehingga angka yang dikutip di dalam narasi menjadi tidak sinkron dengan angka yang tercantum di tabel.

2. Pemanfaatan LLM secara langsung untuk menyusun narasi LED menghadirkan risiko yang tidak dapat ditoleransi dalam konteks dokumen formal. LLM tidak memiliki pemahaman bawaan terhadap rubrik penilaian LAM-Teknik yang spesifik, sehingga narasi yang dihasilkan cenderung generik dan tidak selaras dengan indikator penilaian yang diharapkan asesor. Dalam skenario paling buruk, LLM dapat menghasilkan halusinasi data seperti mencantumkan angka persentase lulusan yang bekerja yang tidak berasal dari tabel LKPS mana pun, atau mengutip indikator rubrik yang tidak ada dalam dokumen panduan LAM-Teknik. Hal tersebut membuat narasi terkesan meyakinkan namun sebenarnya tidak valid [3].
3. Tim penyusun tidak memiliki mekanisme untuk mengetahui kualitas narasi yang sedang disusun sebelum tahap penilaian asesor. Seorang penanggungjawab kriteria yang telah menyelesaikan narasi sub-bab indikator kinerja utama tidak memiliki cara untuk memeriksa apakah paragraf yang ditulisnya sudah memenuhi indikator spesifik rubrik LAM-Teknik atau masih jauh dari skor yang diharapkan. Penilaian baru diketahui setelah asesor memberikan skor pada saat visitasi akan tetapi perbaikan sudah sangat terlambat untuk dilakukan.
4. Proses iterasi penulisan yang panjang rentan menghilangkan formulasi narasi yang sebelumnya dianggap lebih baik. Ketika penanggungjawab kriteria meminta regenerasi narasi untuk ketiga kalinya karena hasil sebelumnya belum memuaskan, tidak ada mekanisme untuk kembali ke formulasi dari iterasi pertama yang sebenarnya lebih tepat dalam menguraikan konteks kebijakan, draf sebelumnya sudah tertimpa dan tidak tersimpan secara terpisah.

1.3 TUJUAN

Berdasarkan permasalahan yang telah dirumuskan, penelitian ini mengajukan rancang bangun sebuah platform berbasis web terintegrasi yang menyediakan lingkungan kerja terpadu untuk penyusunan LED, mencakup antarmuka penulisan berbasis *rich text editor*, manajemen riwayat draf, serta integrasi dengan layanan *Generative AI* eksternal melalui lapisan orkestrasi *prompt*. Adapun tujuan spesifik penelitian ini adalah sebagai berikut.

1. Membangun platform web terintegrasi yang mengatasi masalah diskrepansi data dengan menyediakan antarmuka penulisan yang memungkinkan pengguna merujuk data kuantitatif dari tabel LKPS

- secara langsung di dalam lingkungan kerja yang sama. Platform menunjukkan referensi data di sekitar area penulisan narasi berbasis *rich text editor*, sehingga pengguna dapat memverifikasi angka yang dikutip dan risiko ketidaksesuaian antara data dan narasi dapat ditekan.
2. Merancang lapisan orkestrasi *prompt* melalui perancangan *template prompt* yang terstruktur dan bervariasi sesuai konteks bab, sub-bab, dan sub-sub-bab LED yang sedang aktif. Setiap *template* dirancang agar instruksi yang dikirimkan ke layanan RAG memuat konteks yang presisi sesuai kriteria yang sedang disusun, sehingga narasi yang dihasilkan lebih selaras dengan indikator penilaian rubrik LAM-Teknik yang berlaku untuk bagian tersebut dan minim risiko halusinasi data [3, 12].
 3. Menyediakan fitur umpan balik evaluasi secara *real-time* terhadap narasi yang sedang disusun sebagai respons atas ketiadaan mekanisme penilaian kualitas sebelum tahap visitasi asesor. Fitur ini memanfaatkan paradigma *LLM-as-a-Judge* untuk memberikan simulasi skor dan saran perbaikan yang disampaikan secara bertahap melalui mekanisme *streaming*, sehingga tim penyusun dapat memperbaiki kualitas narasi secara iteratif sebelum dokumen dikirimkan [4].
 4. Mengembangkan sistem manajemen riwayat draf yang adaptif terhadap iterasi penulisan sebagai solusi atas risiko hilangnya formulasi narasi terbaik selama proses regenerasi. Sistem ini memisahkan draf aktif dari *snapshot* yang disimpan secara eksplisit oleh pengguna, sehingga setiap versi draf yang dianggap baik dapat dilacak, dibandingkan, dan dipulihkan kapan saja tanpa terganggu oleh iterasi generasi berikutnya.

1.4 BATASAN DAN RUANG LINGKUP PENELITIAN

Penelitian ini difokuskan pada perancangan dan pembangunan platform web penyusunan LED dari sisi integrasi sistem, yang mencakup lapisan antarmuka pengguna berbasis React, lapisan peladen berbasis NestJS, serta lapisan orkestrasi *prompt* yang bertanggung jawab atas perancangan *template prompt* yang bervariasi sesuai konteks bab, sub-bab, dan sub-sub-bab LED yang sedang aktif. Selain ketiga lapisan tersebut, penelitian ini juga mencakup integrasi platform dengan layanan RAG melalui kontrak API yang telah disepakati, sehingga seluruh kapabilitas generasi narasi dan evaluasi skoring dapat dikonsumsi oleh platform tanpa penulis perlu membangun komponen RAG itu sendiri.

Pengembangan layanan RAG, termasuk komponen *retrieval*, *vector database*, dan lapisan generasi berbasis LLM, berada di luar lingkup penelitian ini dan dikerjakan oleh anggota tim lain dalam proyek akhir terpisah. Selain itu, platform yang dirancang difokuskan pada penggunaan tunggal (*single-user*) sebagai fondasi awal, sehingga mekanisme kolaborasi *real-time* antar anggota tim penyusun tidak termasuk dalam lingkup penelitian ini dan diposisikan sebagai arah pengembangan lanjutan.

1.5 MANFAAT

Penelitian ini diharapkan dapat memberikan kontribusi nyata terhadap proses penjaminan mutu pendidikan tinggi, khususnya pada konteks penyusunan Laporan Evaluasi Diri (LED) untuk akreditasi LAM-Teknik. Manfaat dari sistem yang dirancang dapat dirasakan oleh beberapa pihak dengan bentuk kontribusi yang spesifik sebagai berikut.

1. Bagi tim penyusun LED program studi, platform ini menyediakan lingkungan kerja terpadu yang menggantikan alur kerja terfragmentasi antara aplikasi pengolah angka, pengolah kata, dan dokumen panduan terpisah menjadi satu antarmuka tunggal. Tim penyusun memperoleh kemudahan dalam menghasilkan draf narasi awal, ketersediaan umpan balik evaluasi terhadap kepatuhan rubrik, serta kemampuan melacak dan memulihkan riwayat draf sehingga tidak ada formulasi tulisan yang hilang selama proses iterasi.
2. Bagi program studi dan institusi pendidikan tinggi, platform ini berpotensi meningkatkan kualitas dokumen LED yang diajukan ke LAM-Teknik melalui konsistensi narasi dan kesesuaian dengan rubrik penilaian, yang pada akhirnya dapat berdampak positif pada hasil akreditasi. Selain itu, berkurangnya beban kerja tim penyusun memungkinkan dosen dan staf untuk lebih fokus pada aktivitas tridarma lainnya seperti pengajaran dan penelitian.
3. Bagi pengembangan bidang keilmuan rekayasa perangkat lunak, penelitian ini memberikan kontribusi berupa studi kasus perancangan dan pembangunan platform web yang mengintegrasikan layanan *Generative AI* eksternal sebagai komponen pendukung dalam konteks pembuatan dokumen formal yang menuntut akurasi tinggi. Rancangan platform yang dihasilkan dapat menjadi rujukan bagi pengembangan sistem serupa di domain lain yang membutuhkan integrasi layanan AI ke dalam alur kerja penulisan dokumen terstruktur.

4. Bagi penelitian lanjutan, hasil penelitian ini dapat menjadi pijakan bagi penelitian berikutnya, baik dalam bentuk pengembangan fitur lanjutan pada platform maupun dalam bentuk studi yang mengukur efektivitas platform berbantuan AI dalam meningkatkan kualitas dan efisiensi penyusunan dokumen formal di domain akreditasi dan penjaminan mutu lainnya.

1.6 SISTEMATIKA PENULISAN

Buku proyek akhir ini disusun dalam lima bab dengan sistematika sebagai berikut:

Bab 1 Pendahuluan

Memuat latar belakang, permasalahan, tujuan, batasan dan ruang lingkup penelitian, manfaat, serta sistematika penulisan yang menjadi dasar pelaksanaan penelitian.

Bab 2 Kajian Pustaka

Membahas landasan teori yang digunakan dalam penelitian, meliputi teknologi pendukung pembangunan platform web dan layanan AI yang diintegrasikan, disertai tinjauan terhadap penelitian terdahulu yang relevan.

Bab 3 Desain Sistem

Memaparkan deskripsi solusi, desain antarmuka platform, serta desain sistem yang mencakup arsitektur platform, rancangan basis data, pemodelan fungsionalitas melalui *use case diagram*, dan metrik evaluasi sistem.

Bab 4 Eksperimen dan Analisis

Membahas implementasi sistem, hasil pengujian fungsionalitas, eksperimen variasi *template prompt*, serta analisis kinerja sistem berdasarkan metrik evaluasi yang telah ditetapkan.

Bab 5 Penutup

Berisi kesimpulan dari hasil penelitian beserta saran untuk pengembangan lebih lanjut.

BAB 2

KAJIAN PUSTAKA

Proyek akhir ini berada pada irisan dua bidang, yaitu penjaminan mutu pendidikan tinggi melalui akreditasi LAM-Teknik dan rekayasa perangkat lunak dengan bantuan *Generative AI*. Domain pertama menempatkan Laporan Evaluasi Diri (LED) sebagai dokumen strategis yang menuntut sinkronisasi antara data kuantitatif, narasi analitis, serta kepatuhan terhadap rubrik penilaian yang kompleks. Domain kedua menghadirkan peluang berupa kemampuan *Large Language Model* dalam menghasilkan narasi panjang secara otomatis, sekaligus menghadirkan tantangan teknis baru ketika model tersebut harus diintegrasikan ke dalam aplikasi produksi yang menuntut akurasi tinggi dan kontrol terhadap keluarannya.

Permasalahan teknis utama adalah bagaimana membangun sebuah lingkungan kerja penulisan terpadu berbasis web yang mampu mengelola *state* dokumen kompleks lintas kriteria akreditasi, mengorkestrasikan *prompt* secara terstruktur ke layanan *Generative AI* berbasis *Retrieval-Augmented Generation* (RAG), menyampaikan umpan balik evaluasi secara responsif, serta menjamin persistensi draf di tengah iterasi penulisan dengan bantuan AI. Permasalahan tersebut dirumuskan sebagai integrator sistem yang bertanggung jawab pada lapisan antarmuka pengguna, lapisan orkestrasi *prompt*, dan lapisan persistensi data, sementara mesin RAG itu sendiri dikembangkan secara terpisah oleh anggota tim lain dan diakses melalui kontrak API yang telah disepakati. Selaras dengan batasan yang ditetapkan pada Bab 1, platform yang dirancang berfokus pada penggunaan tunggal (*single-user*) sehingga pembahasan teori penunjang pada bab ini tidak mencakup mekanisme kolaborasi *real-time* maupun penyelesaian konflik penyuntingan simultan.

2.1 DESKRIPSI PERMASALAHAN

Proyek akhir ini berada pada irisan dua bidang, yaitu penjaminan mutu pendidikan tinggi melalui akreditasi LAM-Teknik dan rekayasa perangkat lunak untuk pembangunan platform web yang mengintegrasikan layanan *Generative AI* eksternal. Domain pertama menempatkan Laporan Evaluasi Diri (LED) sebagai dokumen strategis yang menuntut sinkronisasi antara data kuantitatif, narasi analitis, serta kepatuhan terhadap rubrik penilaian yang kompleks. Domain kedua menghadirkan tantangan rekayasa dalam merancang sistem yang mampu mengonsumsi layanan AI eksternal secara terkendali, sekaligus menyajikannya melalui antarmuka yang ergonomis dan andal bagi pengguna.

Permasalahan teknis utama yang dihadapi adalah bagaimana membangun sebuah lingkungan kerja penulisan terpadu berbasis web yang mampu mengelola *state* dokumen kompleks lintas kriteria akreditasi, mengintegrasikan layanan *Generative AI* melalui *template prompt* yang konteks-adaptif, menyampaikan umpan balik evaluasi secara responsif, serta menjamin persistensi draf di tengah iterasi penulisan. Permasalahan tersebut dirumuskan dari sudut pandang integrator sistem yang bertanggung jawab pada lapisan antarmuka pengguna, lapisan orkestrasi *prompt*, dan lapisan persistensi data, sementara layanan RAG itu sendiri dikembangkan secara terpisah oleh anggota tim lain dan diakses melalui kontrak API yang telah disepakati. Selaras dengan batasan yang ditetapkan pada Bab 1, platform yang dirancang berfokus pada penggunaan tunggal (*single-user*) sehingga pembahasan teori penunjang pada bab ini tidak mencakup mekanisme kolaborasi *real-time* maupun penyelesaian konflik penyuntingan simultan.

2.2 TEORI PENUNJANG

Berikut adalah beberapa teori penunjang yang relevan untuk mendukung penelitian proyek akhir ini.

2.2.1 Laporan Evaluasi Diri

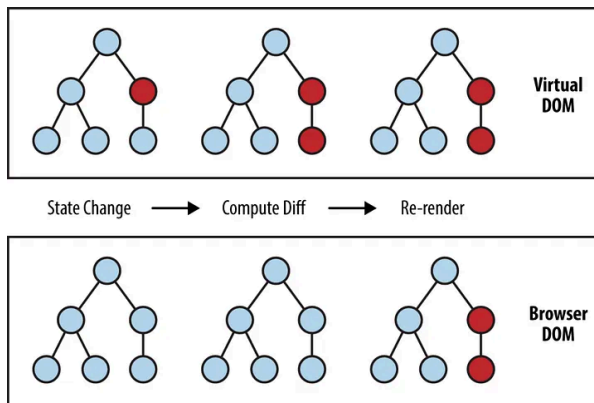
Laporan Evaluasi Diri (LED) merupakan salah satu dokumen akreditasi paling strategis yang disusun oleh internal perguruan tinggi sebagai bentuk refleksi mandiri terhadap mutu penyelenggaraan program studi. Dokumen ini memuat profil program studi, capaian kinerja pada setiap kriteria penilaian, hasil analisis SWOT, serta narasi evaluatif yang menjelaskan bagaimana program studi mengelola sumber daya, menjalankan proses pendidikan, dan menghasilkan luaran sesuai standar yang ditetapkan. Dokumen LED memaparkan kondisi riil program studi sekaligus argumen yang meyakinkan asesor mengenai kualitas penyelenggaraan Tridharma [8].

LED dianggap strategis karena dua alasan utama. Pertama, bobot LED menempati porsi yang sangat menentukan dalam peringkat akreditasi, sehingga kualitas penyusunannya berdampak langsung pada hasil akhir yang diperoleh program studi. Kedua, LED merupakan satu-satunya dokumen yang memadukan data kuantitatif dengan narasi kualitatif analitis dalam satu kesatuan. Tabel kinerja harus diiringi penjelasan naratif yang menafsirkan angka, mengidentifikasi kekuatan dan kelemahan, serta menjelaskan strategi pengembangan ke depan.

Penyusunan dokumen LED dilakukan oleh tim internal program studi yang mengacu pada panduan dan rubrik penilaian dari LAM-Teknik. Setiap kriteria memiliki indikator spesifik dengan deskripsi rubrik yang menjelaskan kriteria untuk mencapai skor pada masing-masing tingkatan penilaian, sehingga penyusunan LED menuntut akurasi data, kemampuan analitis, serta pemahaman mendalam terhadap rubrik yang berlaku.

2.2.2 React.js

React adalah pustaka *JavaScript* sumber terbuka yang dikembangkan oleh Meta, yang dulunya bernama Facebook, untuk membangun antarmuka pengguna berbasis komponen. Konsep utamanya adalah antarmuka aplikasi disusun dari komponen-komponen kecil yang dapat digunakan kembali, masing-masing mengelola tampilan dan perilakunya sendiri. React dapat menggunakan sintaks *JSX* yang memungkinkan penulisan struktur antarmuka secara deklaratif di dalam kode *JavaScript*. React menerapkan mekanisme *virtual DOM* untuk memperbarui tampilan secara efisien dengan hanya merender ulang bagian antarmuka yang benar-benar berubah [6].



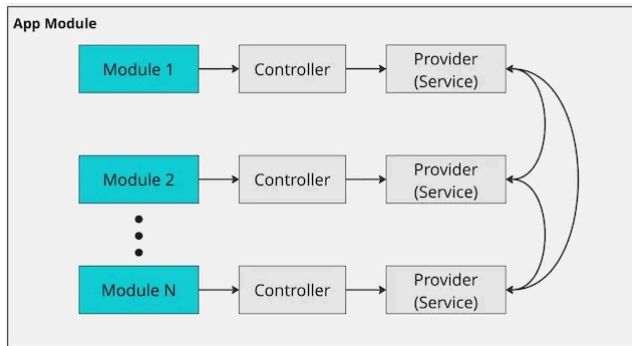
Gambar 2.1. Diagram kolaborasi antar *Virtual DOM* dengan *Browser DOM*.
Sumber: *React.js Virtual DOM: An In-Depth Exploration*, Medium. 2025.

React menjadi pilihan yang sesuai untuk platform penyusunan LED yang memiliki antarmuka kompleks dan dinamis, seperti *rich text editor*, panel umpan balik skoring secara real-time, serta daftar *checkpoint* draf yang perlu diperbarui secara responsif terhadap interaksi pengguna. Dukungan ekosistem React yang luas, seperti integrasi dengan pustaka *rich*

text editor terkini dengan TipTap atau Lexical, pustaka manajemen *state* dengan Zustand atau Redux. Hal tersebut semakin memperkuat justifikasi pemilihan React sebagai pondasi lapisan *frontend* pada proyek akhir ini.

2.2.3 NestJS

NestJS adalah *framework* sumber terbuka untuk membangun aplikasi sisi *server* berbasis *Node.js* yang dirancang dengan dukungan *TypeScript* secara native. NestJS menerapkan arsitektur modular yang terinspirasi dari pola *Model-View-Controller* dan konsep *dependency injection*. Setiap fitur aplikasi dipecah ke dalam modul-modul yang berisi *controller*, *service*, dan *provider* dengan tanggung jawab masing-masing yang jelas. Pemisahan ini memudahkan pengembang dalam mengelola kompleksitas aplikasi, melakukan pengujian, serta memperluas fungsionalitas tanpa harus mengubah struktur kode yang sudah ada [10].



Gambar 2.2. Diagram pola *dependency injection* antar modul.

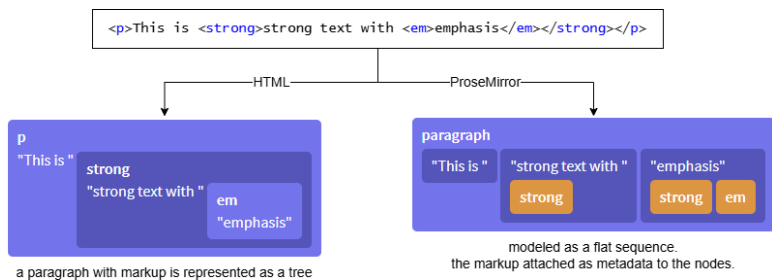
Sumber: *Basic Concepts of Nest JS in SUPER Detail*, Medium. 2025.

NestJS dipilih sebagai lapisan *backend* pada platform penyusunan LED yang menuntut struktur kode yang teratur dan dapat dikembangkan secara bertahap. Modul-modul terpisah dapat digunakan untuk menangani tanggung jawab seperti autentikasi pengguna, pengelolaan draf, lapisan orkestrasi *prompt* yang berkomunikasi dengan layanan RAG, serta penyaluran umpan balik skoring secara real-time. Dukungan native terhadap *TypeScript* juga menjamin konsistensi tipe data antara sisi *frontend* dan *backend* yang sama-sama menggunakan *TypeScript*, sehingga mengurangi risiko *runtime error* yang berasal dari ketidaksesuaian kontrak data antar lapisan.

2.2.4 Rich Text Editor

Rich text editor adalah komponen antarmuka yang memungkinkan pengguna menulis dan memformat teks secara langsung di dalam halaman web. Hasil pemformatan dapat langsung terlihat tanpa perlu menulis sintaks pemformatan secara manual. Perbedaan paling jelas antara *plain text editor*, yang hanya menerima karakter biasa, dengan *rich text editor* adalah dukungan berbagai elemen seperti judul, paragraf, daftar, tabel, kutipan, gaya teks tebal dan miring, hingga penyisipan elemen seperti gambar atau tautan. Kemampuan tersebut menjadi kebutuhan dasar dalam laporan LED, mengingat narasi yang ditulis bersifat panjang, berstruktur, dan memuat elemen pemformatan yang kompleks.

Pengembangan *rich text editor* pada peramban web pada awalnya bergantung pada atribut *contenteditable* bawaan HTML yang memiliki keterbatasan serius karena perilakunya bervariasi antar peramban, struktur HTML yang dihasilkan tidak konsisten, dan sulit dikontrol secara programatik. Pendekatan *structured document model* muncul sebagai jawaban atas masalah tersebut dengan memisahkan representasi dokumen dari tampilan visualnya. Dokumen direpresentasikan dalam bentuk pohon JSON yang menggambarkan hierarki elemennya dan tampilan visual hanyalah hasil render dari struktur data tersebut. Pendekatan ini diterapkan oleh berbagai *toolkit* dan pustaka modern, di antaranya ProseMirror sebagai *toolkit* tingkat rendah yang menjadi fondasi banyak editor modern, serta TipTap sebagai *framework* tingkat tinggi yang dibangun di atas ProseMirror dengan ekstensi siap pakai, yang dipilih sebagai *framework rich text editor* pada proyek akhir ini [15, 11].



Gambar 2.3. Perbandingan HTML dan ProseMirror dalam mempresentasikan *markup*.

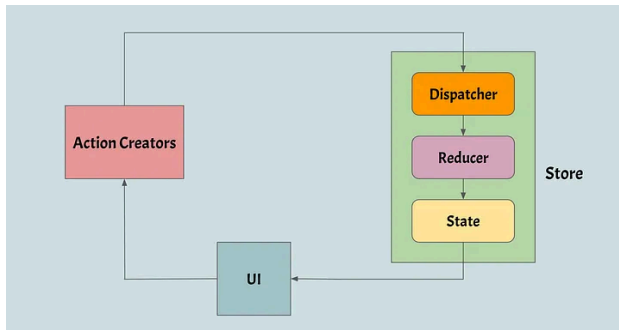
Sumber: ProseMirror Guide. 2026.

Pemilihan TipTap didasarkan pada beberapa pertimbangan teknis yang selaras dengan kebutuhan platform penyusunan LED. Pada konteks platform ini, *rich text editor* harus mampu berinteraksi dengan komponen-komponen sistem lainnya secara mendalam, seperti menerima sisipan narasi hasil generasi AI serta menjadi sumber data yang konsisten untuk mekanisme *auto-save* dan draf berbasis *snapshot*. Representasi dokumen dalam bentuk JSON yang dihasilkan oleh *structured document model* sangat mendukung kebutuhan tersebut karena struktur dokumen dapat disimpan, dimanipulasi, dan dipulihkan tanpa kehilangan informasi pemformatan. TipTap juga memiliki integrasi yang sangat baik dengan React melalui paket resmi yang menyediakan *hooks* dan komponen siap pakai, sehingga *state editor* dapat dikelola secara natural di dalam pohon komponen React tanpa perlu lapisan adaptasi tambahan. Di atas keunggulan tersebut, TipTap juga menawarkan dukungan komunitas yang aktif dan sistem ekstensi yang robust, sehingga memungkinkan pengembangan ekstensi khusus untuk integrasi *AI content generation* secara lebih cepat dan akurat.

2.2.5 State Management

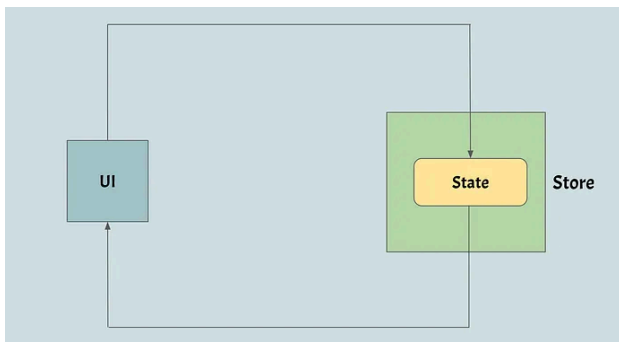
State management adalah praktik pengelolaan data di sisi *frontend* yang menentukan bagaimana data disimpan, diakses, diperbarui, dan disinkronkan antar komponen sepanjang siklus hidup aplikasi. Terdapat dua kategori *state* utama pada aplikasi modern yang kompleks. Pertama, *client state* yang merepresentasikan data antarmuka pengguna seperti isi *form*, status modal, dan kondisi *editing*. Kedua, *server state* yang merepresentasikan data yang berasal dari *backend* dan perlu disinkronkan dengan sumber kebenaran di sisi *server*. Pemisahan kedua kategori tersebut menjadi penting karena karakteristik pengelolaannya berbeda. *client state* membutuhkan akses cepat dan pembaruan reaktif terhadap interaksi pengguna, sementara *server state* membutuhkan mekanisme *caching*, *invalidation*, dan sinkronisasi otomatis dengan *backend*.

Dalam pengembangan aplikasi modern menggunakan React, terdapat beberapa pustaka *state management* yang banyak digunakan seperti Zustand, dan TanStack Query yang sebelumnya dikenal sebagai React Query. Zustand adalah pustaka *state management* minimalis berbasis *hooks* yang menyediakan *store* dengan API sederhana dan boilerplate yang rendah dibandingkan kompetitor nya seperti Redux [18].



Gambar 2.4. Diagram arsitektur Redux.

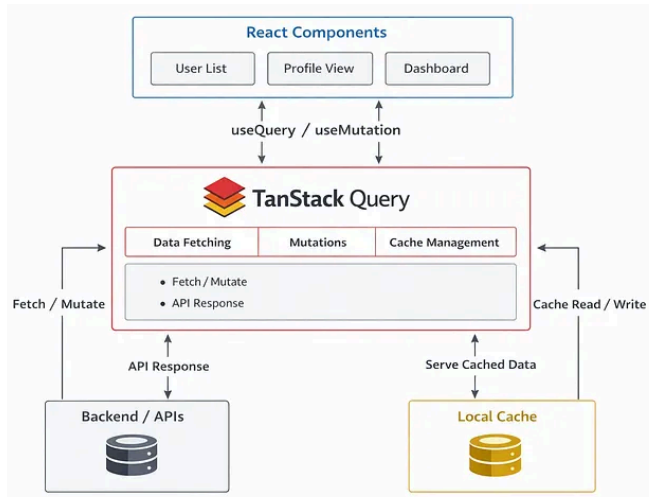
Sumber: *React State Management — using Zustand*, Medium. 2024.



Gambar 2.5. Diagram arsitektur Zustand.

Sumber: *React State Management — using Zustand*, Medium. 2024.

TanStack Query memiliki fokus berbeda karena khusus menangani *server state* dengan menyediakan mekanisme *caching* otomatis berbasis *query key*, *background refetching*, *retry logic*, serta *invalidation* yang terstruktur, sehingga *developer* tidak perlu menulis logika sinkronisasi data secara manual [14].



Gambar 2.6. Diagram arsitektur TanStack Query.

Sumber: *React Data Fetching Best Practices: Why TanStack Query is Essential for Scaling*, Medium. 2026.

Platform penyusunan LED memiliki karakteristik *state management* yang khas. Setiap bab dalam laporan memiliki *state* tersendiri yang dapat berpindah secara dinamis ketika pengguna berganti konteks bab, sekaligus harus mengirim identifier bab yang sedang aktif ke layanan *backend* untuk memastikan layanan RAG memproses permintaan pada konteks yang tepat. Sementara di bagian lain terdapat sejumlah *side effect* berjalan secara asinkron seperti *auto-save* di latar belakang, sinkronisasi draf, dan permintaan generasi narasi. Karakteristik tersebut menjadi dasar pemilihan kombinasi Zustand dan TanStack Query. Zustand bertanggung jawab atas *client state* yang berkaitan dengan interaksi pengeditan secara lokal dan butuh feedback cepat, sedangkan TanStack Query bertanggung jawab atas *server state* dengan *cache* yang otomatis terpisah per identifier bab sehingga data tetap tersedia ketika pengguna berpindah antar bab tanpa perlu permintaan ulang ke *backend*.

2.2.6 Generative AI dan Large Language Model (LLM)

Generative AI adalah cabang kecerdasan buatan yang berfokus pada pembuatan konten baru berdasarkan pola yang dipelajari dari data pelatihan. Kategori yang paling relevan untuk platform ini adalah *Large Language Model* (LLM), yaitu model bahasa berskala besar yang mampu

menghasilkan narasi panjang yang koheren sesuai instruksi pengguna. Meskipun kemampuan generasi narasi LLM sangat menjanjikan, pemanfaatannya secara langsung pada dokumen formal seperti LED menghadirkan risiko halusinasi data, kondisi ketika model menghasilkan informasi yang tampak meyakinkan namun tidak valid secara faktual, serta ketidakselarasan narasi dengan rubrik penilaian yang spesifik karena LLM tidak memiliki pemahaman bawaan terhadap konteks akreditasi. Karakteristik ini menjadi dasar pertimbangan digunakannya layanan *Retrieval-Augmented Generation* (RAG) sebagai komponen AI eksternal pada platform, sebagaimana diuraikan pada sub-bab berikutnya.

2.2.7 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) adalah pendekatan arsitektural yang memperkaya keluaran LLM dengan informasi kontekstual yang diambil dari sumber pengetahuan eksternal sebelum model menghasilkan respons. Pendekatan ini mengatasi keterbatasan LLM standar yang rentan terhadap halusinasi data dengan cara menyisipkan potongan informasi yang relevan dari basis pengetahuan ke dalam *prompt* sebagai konteks tambahan, sehingga keluaran model lebih akurat dan terikat pada sumber yang dapat diverifikasi [3]. Pada konteks penyusunan LED, basis pengetahuan layanan RAG diisi dengan dokumen panduan dan rubrik penilaian LAM-Teknik sehingga setiap permintaan generasi narasi secara otomatis diperkaya dengan kutipan rubrik yang relevan. Pada proyek akhir ini, layanan RAG dikembangkan secara terpisah oleh anggota tim lain, sementara platform yang dibangun berperan sebagai konsumen yang mengakses kapabilitas generasi dan evaluasi narasi tersebut melalui kontrak API yang telah disepakati.

2.2.8 Prompt Engineering

Prompt engineering adalah praktik merancang dan mengoptimalkan instruksi yang dikirim ke *Large Language Model* agar menghasilkan keluaran yang sesuai dengan ekspektasi, melalui aktivitas sistematis yang melibatkan perancangan struktur instruksi yang efektif dan iterasi yang terukur untuk mencapai kualitas keluaran yang diinginkan [12]. Pada konteks penyusunan LED, *prompt engineering* menjadi kritis karena keluaran LLM tidak cukup hanya koheren, tetapi juga harus akurat secara faktual, selaras dengan rubrik penilaian LAM-Teknik, dan menggunakan bahasa formal akademis sesuai standar dokumen akreditasi. Sebuah *template prompt* yang berkualitas umumnya tersusun dari beberapa elemen yang saling melengkapi, seperti instruksi sistem yang menetapkan peran dan

tujuan model, konteks pendukung yang menyediakan informasi latar belakang yang relevan, instruksi tugas yang menjelaskan apa yang diharapkan dihasilkan, serta spesifikasi format keluaran yang memudahkan pemrosesan lebih lanjut oleh aplikasi.

Pada platform ini, *prompt engineering* diwujudkan melalui perancangan *template prompt* yang bervariasi sesuai hierarki LED yang sedang aktif. Setiap bab, sub-bab, dan sub-sub-bab memiliki karakteristik penilaian yang berbeda-beda dalam rubrik LAM-Teknik, sehingga *template* yang digunakan perlu disesuaikan agar instruksi yang dikirimkan ke layanan RAG memuat konteks yang presisi untuk bagian tersebut. Setiap *template* menggabungkan beberapa teknik secara komplementer: *role-based prompting* untuk menetapkan persona penulis laporan akreditasi [7], *context injection* untuk menyisipkan data kuantitatif pengguna sebagai rujukan faktual, *few-shot prompting* untuk mengarahkan pola bahasa yang diharapkan [1], serta *chain-of-thought prompting* untuk memandu penalaran bertahap sebelum narasi akhir disusun [16]. Pengaturan *temperature* pada nilai rendah juga diterapkan untuk menjaga konsistensi keluaran antar permintaan yang serupa.

Pengelolaan *template prompt* pada platform ini dilakukan menggunakan Langfuse, sebuah platform manajemen *prompt* dan observabilitas LLM yang memungkinkan penyimpanan, pengelolaan versi, dan pembaruan *template* secara terpusat tanpa perlu mengubah kode peladen [19]. Dengan pendekatan ini, iterasi dan penyempurnaan *template* dapat dilakukan secara independen dari siklus pengembangan aplikasi, sehingga kualitas keluaran narasi dapat ditingkatkan secara berkelanjutan.

2.2.9 Skoring Otomatis Berbasis LLM

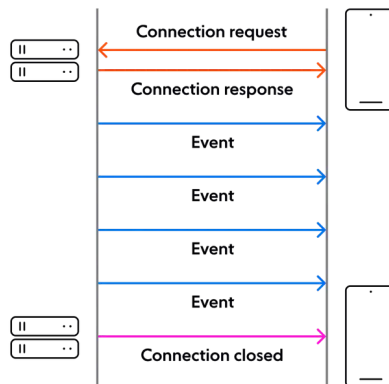
Penilaian kualitas narasi LED secara otomatis membutuhkan pendekatan yang lebih dari sekadar metrik tekstual klasik karena penilaian akreditasi bersifat semantik dan terikat pada rubrik LAM-Teknik yang spesifik. Paradigma yang digunakan untuk kebutuhan tersebut adalah *LLM-as-a-Judge*, di mana *Large Language Model* dilibatkan sebagai evaluator yang menilai kualitas teks berdasarkan kriteria yang diberikan dalam bentuk *prompt* evaluasi, dengan tingkat korelasi yang tinggi terhadap penilaian manusia ahli ketika rubrik disusun secara terstruktur [4]. Kerangka evaluasi yang selaras dengan paradigma ini adalah *Retrieval-Augmented Generation Assessment* (RAGAS), sebuah pendekatan evaluasi yang mengukur kualitas keluaran sistem RAG melalui empat metrik utama: *faithfulness* yang mengukur sejauh mana narasi yang dihasilkan berpijak pada konteks yang diambil, *context recall* yang mengukur kelengkapan konteks yang berhasil diambil dari basis

pengetahuan, *answer relevancy* yang mengukur relevansi narasi terhadap kriteria yang dievaluasi, serta *answer precision* yang mengukur efisiensi pemanfaatan konteks yang tersedia dalam membentuk narasi akhir.

Pada platform ini, keempat metrik RAGAS tersebut ditampilkan kepada pengguna dalam bentuk yang lebih mudah dipahami dalam konteks penyusunan LED. *Faithfulness* ditampilkan sebagai akurasi bukti dan fakta, *context recall* ditampilkan sebagai kelengkapan panduan, *answer relevancy* ditampilkan sebagai kesesuaian rubrik, dan *answer precision* ditampilkan sebagai efisiensi rujukan. Selain keempat metrik tersebut, platform juga menampilkan saran perbaikan yang dikategorikan berdasarkan tingkat prioritas untuk membantu pengguna menindaklanjuti hasil evaluasi secara konkret. Kapabilitas evaluasi ini diakses melalui layanan RAG dan diintegrasikan sebagai fitur umpan balik dalam alur kerja platform.

2.2.10 Server-Sent Events (SSE)

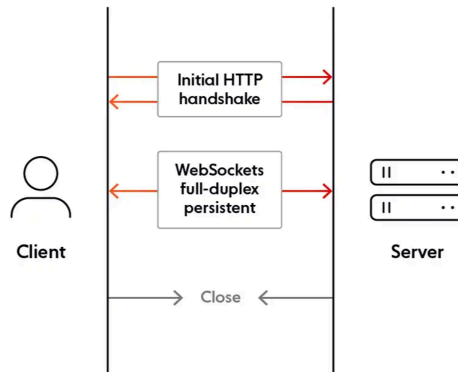
Server-Sent Events (SSE) adalah teknologi komunikasi berbasis HTTP yang memungkinkan *server* mengirim pembaruan data ke *client* secara *streaming* melalui satu koneksi yang tetap terbuka. REST konvensional berbeda dengan komunikasi SSE karena bersifat *request-response* dan menutup koneksi setelah respons dikirim. SSE mempertahankan koneksi HTTP terbuka agar *server* dapat terus mendorong (*push*) data baru ke *client* selama koneksi masih aktif. Setiap pesan yang dikirim melalui SSE disusun dalam format teks sederhana yang dapat langsung dikonsumsi oleh peramban modern melalui antarmuka *EventSource* yang sudah tersedia secara *native* tanpa memerlukan pustaka tambahan.



Gambar 2.7. Diagram pola komunikasi SSE.

Sumber: WebSockets vs Server-Sent Events: Key differences and which to use in 2024, ably. 2023.

WebSocket juga merupakan teknologi komunikasi *real-time*. Perbedaanannya adalah SSE memiliki karakteristik yang lebih sederhana namun terbatas pada komunikasi satu arah dari *server* ke *client*. Sementara *WebSocket* mendukung komunikasi dua arah penuh (*full-duplex*), *client* dan *server* dapat saling mengirim pesan kapan saja.



Gambar 2.8. Diagram pola komunikasi WebSocket.

Sumber: *WebSockets vs Server-Sent Events: Key differences and which to use in 2024*, ably. 2023.

Websocket lebih sesuai untuk skenario interaktif seperti aplikasi *chat* atau permainan daring. Sementara SSE dirancang khusus untuk skenario di mana *server* hanya perlu mendorong pembaruan ke *client* tanpa membutuhkan respons aktif dari *client*. SSE juga berjalan di atas protokol HTTP standar sehingga lebih mudah diintegrasikan dengan infrastruktur *web*, lebih ramah terhadap *firewall* dan *proxy*, serta memiliki mekanisme *reconnect* otomatis ketika koneksi terputus. *WebSocket* tidak menyediakan mekanisme *reconnect* otomatis secara *native*, sehingga implementasinya perlu ditambahkan secara manual oleh pengembang.

Pada platform penyusunan LED, SSE dipilih sebagai mekanisme komunikasi *real-time* untuk kebutuhan *streaming* hasil generasi narasi dari layanan RAG dan penyampaian umpan balik skoring secara responsif. Kebutuhan tersebut memiliki pola komunikasi yang sama. Pada *streaming* hasil generasi, narasi yang dihasilkan oleh LLM dapat dikirim *token* per *token* ke antarmuka pengguna seiring dimunculkan oleh model agar pengguna tidak perlu menunggu seluruh narasi selesai untuk melihat hasil.

Pada umpan balik skoring, skor dan rekomendasi perbaikan dapat dikirim secara bertahap ketika sistem RAG menyelesaikan evaluasi untuk masing-masing aspek penilaian. Karakteristik satu arah dari SSE sangat sesuai dengan kedua skenario tersebut, sekaligus menghadirkan implementasi yang lebih ringan dan lebih mudah dikelola dibandingkan *WebSocket* dalam konteks ini.

2.2.11 PostgreSQL

PostgreSQL adalah sistem manajemen basis data relasional sumber terbuka yang dikenal karena kestabilan, kepatuhan terhadap standar SQL, serta dukungan terhadap fitur lanjutan seperti transaksi ACID, *foreign key constraint*, *indexing* multi-kolom, hingga ekstensibilitas yang memungkinkan penambahan tipe data. Properti ACID (*Atomicity*, *Consistency*, *Isolation*, *Durability*) sangat berguna untuk operasi penyimpanan draf pada aplikasi penyusunan LED, karena melibatkan perubahan beberapa data sekaligus sehingga harus dieksekusi secara atomik. Misalnya pembaruan draf konten yang harus disertai dengan pencatatan metadata terkait agar konsistensi data tetap terjaga.

PostgreSQL memiliki tipe data JSONB yang sangat relevan dengan kebutuhan platform penyusunan LED. Tipe JSONB memungkinkan basis data menyimpan dokumen JSON secara langsung dalam kolom tabel dan dioptimalkan dalam bentuk biner sambil tetap mendukung *indexing* melalui *Generalized Inverted Index* (GIN) untuk pencarian dalam struktur JSON, serta validasi tipe pada level basis data. Hal tersebut sesuai dengan penyimpanan konten *rich text editor* yang direpresentasikan dalam bentuk pohon JSON yang menggambarkan hierarki elemen-elemen dokumen. Dengan tipe data JSONB, struktur dokumen dapat disimpan secara utuh tanpa kehilangan informasi pemformatan, mudah dipulihkan kembali, dan tetap dapat dimanipulasi atau dikueri secara terstruktur ketika diperlukan.

Kombinasi antara karakteristik relasional yang kuat dan dukungan tipe data JSONB menjadikan PostgreSQL sebagai pilihan yang tepat. Struktur data pada platform ini memiliki sifat relasional yang jelas dan konten *draft* itu sendiri bersifat semi-terstruktur dan paling efisien disimpan dalam bentuk JSON. PostgreSQL menyediakan keduanya dalam satu sistem basis data sehingga aplikasi tidak perlu memadukan dua sistem berbeda seperti basis data relasional dan basis data dokumen, dan tetap memperoleh keuntungan dari transaksi ACID, kemampuan kueri yang ekspresif, serta ekosistem pustaka *Object-Relational Mapping* yang matang di sisi NestJS.

2.2.12 Manajemen Draf Berbasis Draft History

Proses penyusunan LED bersifat iteratif karena ada serangkaian siklus pengeditan, penyimpanan, dan regenerasi narasi hingga hasil yang diperoleh dianggap memadai. Versi narasi sebelumnya yang mungkin memiliki kualitas baik dapat tergantikan atau hilang jika pengguna meminta regenerasi narasi atau merevisi draf secara manual. Proses iterasi seperti ini perlu mekanisme yang memungkinkan pengguna untuk menyimpan titik-titik yang dianggap baik (*good checkpoint*) dan kemudian dapat mengembalikan ke titik-titik tersebut ketika pengguna ingin membandingkan, memilih, atau melanjutkan dari versi yang berbeda.

Draft history adalah mekanisme penyimpanan yang memisahkan antara *draft aktif*, yang merupakan tempat pengguna melakukan pengeditan, dan *draft yang disimpan*, yang merupakan hasil dari aksi pengguna untuk menyimpan secara eksplisit. Pengguna dapat menyimpan draf dengan memberikan label setiap kali pengguna menyelesaikan penyuntingan. Aksi penyimpanan menciptakan *snapshot* dari konten *draft aktif* pada saat tersebut. Kemudian *snapshot* disimpan sebagai item terpisah dalam *draft history* dan dapat direferensikan kembali kapan saja. Keunikan mekanisme ini adalah pengguna dapat terus mengedit *draft aktif* dan pengguna dapat memilih untuk memuat salah satu draf yang pernah disimpan. *Draft aktif* akan direset dengan konten dari draf yang dipilih dan memungkinkan pengguna melanjutkan pekerjaan dari titik tersebut atau membandingkan dengan versi lain.

Implementasi *draft history* perlu desain skema basis data relasional yang memisahkan data *draft aktif* dan data riwayat draf yang disimpan. Entitas *draft aktif* disimpan dalam struktur tabel yang mudah diakses untuk operasi pembacaan dan penulisan. Entitas *Snapshot* yang akan disimpan dari *draft aktif* disalin ke tabel riwayat dengan metadata seperti label yang diberikan pengguna, waktu penyimpanan, dan urutan penyimpanan. Struktur konten draf yang berbentuk pohon JSON dapat disimpan langsung dalam kolom JSONB pada kedua tabel tersebut. Entitas *draft aktif* digunakan pada mekanisme *auto-save* yang berjalan di latar belakang. Mekanisme ini bertujuan untuk mencegah kehilangan data saat pengguna melakukan perubahan dengan menyimpannya ke *draft aktif* ketika koneksi terputus atau aplikasi crash, bukan membuat item baru di *draft history*. Pengguna perlu secara eksplisit memilih untuk membuat item di *draft history* ketika merasa versi saat ini layak disimpan.

2.3 PENELITIAN TERKAIT

Tinjauan terhadap penelitian terdahulu pada bagian ini dikelompokkan ke dalam empat kategori yang masing-masing relevan dengan platform yang diusulkan. Kategori pertama meninjau pemanfaatan LLM untuk

generasi dokumen formal sebagai latar belakang teknologi AI yang diintegrasikan. Kategori kedua meninjau penerapan RAG sebagai layanan yang mendasari kapabilitas generasi dan evaluasi pada platform ini. Kategori ketiga, yang paling sentral bagi penelitian ini, meninjau perancangan platform penulisan berbasis AI sebagai referensi langsung dalam merancang lingkungan kerja penyusunan LED. Kategori keempat meninjau evaluasi otomatis berbasis rubrik sebagai justifikasi pemilihan paradigma penilaian yang diintegrasikan. Bagian akhir tinjauan menyajikan tabel analisis kesenjangan yang membandingkan cakupan penelitian terdahulu dengan cakupan proyek akhir ini.

2.3.1 Pemanfaatan LLM untuk Generasi Dokumen Administratif

Pemanfaatan *Large Language Model* untuk membantu pengelolaan dokumen administratif telah menjadi area penelitian yang berkembang pesat dalam beberapa tahun terakhir. Syaila et al. melakukan tinjauan literatur sistematis terhadap implementasi kecerdasan buatan untuk verifikasi dokumen, yang menunjukkan bahwa berbagai pendekatan berbasis AI telah diadopsi secara luas untuk mendukung pengelolaan dokumen administratif di berbagai sektor [13]. Tinjauan tersebut memperkuat legitimasi penggunaan AI dalam konteks dokumen formal yang menuntut tingkat akurasi tinggi, namun cakupannya terbatas pada aspek verifikasi dan validasi dokumen yang telah ada, bukan pada generasi narasi dokumen baru secara otomatis.

Pada sisi generasi dokumen, Auer et al. mengusulkan kerangka *multi-agent* berbasis LLM yang memadukan *prompt engineering* dengan *semantic retrieval* untuk menghasilkan dokumen semi-terstruktur dalam domain administrasi publik [9]. Kerangka tersebut memanfaatkan beberapa *agent* LLM yang bekerja secara terkoordinasi untuk memahami template dokumen, mengekstraksi struktur data yang dibutuhkan, dan menyusun keluaran teks yang sesuai dengan format yang diharapkan. Pendekatan tersebut secara konseptual memiliki kesamaan dengan penelitian ini dalam hal pemanfaatan AI untuk mendukung penyusunan dokumen formal, namun fokusnya terbatas pada dokumen administratif berstruktur kaku tanpa mempertimbangkan lingkungan kerja penulisan terpadu, kepatuhan terhadap rubrik penilaian terstruktur, mekanisme penyimpanan iterasi draf, maupun penyediaan umpan balik evaluatif secara *real-time* kepada penyusun dokumen.

2.3.2 Retrieval-Augmented Generation untuk Mengurangi Halusinasi

Pendekatan *Retrieval-Augmented Generation* (RAG) telah berkembang menjadi salah satu solusi paling banyak diadopsi untuk mengatasi keterbatasan LLM dalam menghasilkan keluaran yang akurat secara faktual. Gao et al. melakukan tinjauan komprehensif terhadap perkembangan RAG yang mengklasifikasikan pendekatan tersebut ke dalam tiga paradigma utama, yaitu *Naive RAG*, *Advanced RAG*, dan *Modular RAG*, dengan masing-masing memiliki karakteristik dalam hal kompleksitas mekanisme *retrieval*, strategi *re-ranking*, dan integrasi dengan tahap *generation* [3]. Tinjauan tersebut menegaskan bahwa RAG terbukti efektif dalam menekan halusinasi data pada berbagai domain yang menuntut akurasi tinggi, sekaligus menjadi landasan teoretis yang banyak diadopsi pada penelitian-penelitian terapan setelahnya.

Pada konteks penerapan RAG di domain pendidikan tinggi Indonesia, Hidayat et al. mengimplementasikan *chatbot* berbasis RAG yang diintegrasikan dengan platform Telegram untuk melayani permintaan informasi mahasiswa, dengan basis pengetahuan yang berisi data akademik institusi [5]. Hasil penelitian tersebut menunjukkan bahwa pendekatan RAG mampu memberikan jawaban yang akurat dan relevan terhadap pertanyaan-pertanyaan mahasiswa, sekaligus memvalidasi efektivitas RAG di domain perguruan tinggi Indonesia. Namun, penerapan tersebut terbatas pada pola interaksi *chatbot* berbentuk tanya-jawab singkat, sehingga belum mengeksplorasi pemanfaatan RAG untuk generasi narasi panjang yang berstruktur, terikat pada rubrik penilaian formal, serta perlu disisipkan ke dalam dokumen yang sedang disunting oleh pengguna sebagaimana karakteristik penyusunan LED.

2.3.3 Platform Penulisan dengan Bantuan AI

Perancangan platform penulisan dengan bantuan AI telah menjadi area penelitian yang aktif sejak kemampuan generasi narasi LLM mencapai tingkat yang memadai untuk membantu produktivitas penulis. Coenen et al. mengembangkan *Wordcraft*, sebuah editor kolaboratif manusia-AI untuk penulisan cerita yang memanfaatkan teknik *few-shot learning* untuk mendukung beragam interaksi penulis dengan model bahasa, seperti melanjutkan teks, menyisipkan paragraf di tengah dokumen, serta menyarankan alternatif gaya bahasa [2]. *Wordcraft* memperkenalkan paradigma bahwa LLM dapat berperan bukan sebagai generator otomatis penuh, melainkan sebagai asisten interaktif yang merespons permintaan pengguna pada konteks lokal di dalam editor. Pendekatan tersebut menjadi rujukan penting dalam perancangan antarmuka platform penulisan dengan

bantuan AI, namun konteks penggunaannya berorientasi pada penulisan kreatif tanpa kepatuhan terhadap rubrik penilaian formal serta tanpa mekanisme penyimpanan riwayat draf yang sistematis.

Penelitian yang lebih mutakhir oleh tim TreeWriter mengusulkan editor untuk penulisan dokumen panjang berbasis representasi hierarkis berbentuk pohon, di mana setiap simpul mewakili tingkat abstraksi dokumen yang berbeda mulai dari kerangka tingkat tinggi hingga isi paragraf pada tingkat terendah [17]. Asisten AI yang terintegrasi dengan editor tersebut dapat memberikan saran yang konsisten lintas tingkat hierarki, sehingga membantu penulis menjaga koherensi antara perencanaan struktur dan detail isi pada dokumen panjang. Pendekatan tersebut menunjukkan kematangan komunitas riset dalam merancang platform penulisan dengan bantuan AI untuk dokumen panjang, namun cakupan TreeWriter tetap berada pada konteks penulisan umum dan riset, sehingga belum mengintegrasikan komponen penting yang relevan untuk konteks LED seperti penyisipan kutipan rubrik penilaian sebagai konteks generasi, evaluasi kepatuhan narasi terhadap rubrik secara *real-time*, serta mekanisme penyimpanan iterasi draf yang sistematis untuk mendukung proses regenerasi dengan bantuan AI.

2.3.4 Evaluasi Otomatis Berbasis LLM dengan Rubrik

Evaluasi otomatis terhadap kualitas teks yang dihasilkan model bahasa merupakan area penelitian yang berkembang seiring meningkatnya kebutuhan untuk menilai keluaran LLM secara terstruktur tanpa harus selalu mengandalkan penilai manusia. Hashemi et al. mengusulkan *LLM-Rubric*, yaitu kerangka evaluasi otomatis multi-dimensional yang memanfaatkan LLM sebagai evaluator dengan menggunakan rubrik berisi sejumlah dimensi penilaian, di mana setiap dimensi diukur melalui pertanyaan rubrik spesifik kepada LLM dan hasilnya dikalibrasi dengan jaringan saraf kecil untuk menyelaraskan keluaran model dengan penilaian manusia [4]. Pendekatan tersebut menunjukkan bahwa LLM dapat berperan sebagai *judge* yang menilai keluaran teks pada beberapa dimensi sekaligus, dengan tingkat korelasi yang tinggi terhadap penilaian manusia ahli ketika rubrik disusun secara terstruktur dan terkalibrasi. Pada konteks platform penyusunan LED, paradigma rubric-based evaluation yang serupa relevan untuk diterapkan dalam menilai kepatuhan narasi terhadap rubrik LAM-Teknik, namun penelitian Hashemi et al. memfokuskan implementasi pada domain evaluasi sistem dialog tanya-jawab dan tidak menyentuh aspek integrasi evaluasi tersebut ke dalam alur penyusunan dokumen formal yang iteratif berbasis platform penulisan.

Perbandingan penelitian serupa dengan penelitian yang diusulkan dapat dilihat pada tabel 2.1 berikut ini:

Tabel 2.1. Perbandingan penelitian

No	Judul Penelitian	Masalah yang Diangkat	Fitur Utama
1	Tinjauan Literatur Sistematis: Analisis Implementasi Kecerdasan Buatan untuk Verifikasi Dokumen [13]	Pengelolaan dokumen administratif yang menuntut akurasi tinggi dan validasi sistematis	Tinjauan komprehensif terhadap implementasi AI untuk verifikasi dan validasi dokumen formal
2	LLM-Based Multi-Agent Generation of Semi-structured Documents from Semantic Templates in Public Administration [9]	Generasi dokumen administrasi publik yang berstruktur kaku secara otomatis	Kerangka <i>multi-agent</i> berbasis LLM dengan <i>prompt engineering</i> dan <i>semantic retrieval</i> untuk menghasilkan dokumen semi-terstruktur
3	Optimalisasi Layanan Sistem Informasi Mahasiswa dengan Integrasi Telegram: Chatbot RAG berbasis LLM [5]	Pelayanan informasi mahasiswa di perguruan tinggi yang tidak responsif terhadap kebutuhan tanya-jawab spesifik	<i>Chatbot</i> berbasis RAG yang terintegrasi dengan Telegram untuk melayani permintaan informasi akademik
4	Retrieval-Augmented Generation for Large Language Models: A Survey [3]	Keterbatasan LLM dalam menghasilkan keluaran yang akurat secara faktual dan rentan terhadap halusinasi data	Tinjauan komprehensif terhadap paradigma RAG dengan klasifikasi <i>Naive</i> ,

			<i>Advanced, dan Modular RAG</i>
5	Wordcraft: A Human-AI Collaborative Editor for Story Writing [2]	Keterbatasan interaksi penulis dengan model bahasa pada penulisan kreatif yang membutuhkan kontrol pada konteks lokal	Editor kolaboratif manusia-AI dengan <i>few-shot learning</i> untuk mendukung beragam interaksi penulisan kreatif
No	Judul Penelitian	Masalah yang Diangkat	Fitur Utama
6	TreeWriter: AI-Assisted Hierarchical Planning and Writing for Long-Form Documents [17]	Kesulitan menjaga koherensi antara perencanaan struktur dan detail isi pada penulisan dokumen panjang	Editor berbasis representasi pohon hierarkis dengan asisten AI yang konsisten lintas tingkat abstraksi
7	LLM-Rubric: A Multidimensional, Calibrated Approach to Automated Evaluation of Natural Language Texts [4]	Kebutuhan untuk menilai keluaran teks LLM secara multi-dimensional tanpa selalu mengandalkan penilai manusia	Kerangka evaluasi otomatis berbasis rubrik multi-dimensional dengan kalibrasi terhadap penilaian manusia
8	Proyek Akhir Ini	Fragmentasi alur kerja penyusunan LED yang memaksa tim menggunakan kombinasi aplikasi terpisah, serta ketiadaan lingkungan kerja terpadu yang mengintegrasikan penulisan narasi, referensi kontekstual, evaluasi rubrik, dan manajemen iterasi draf dalam satu platform	Platform web terpadu yang memadukan antarmuka penulisan berbasis <i>rich text editor</i> TipTap, <i>template prompt</i> konteks-adaptif per hierarki LED, mekanisme <i>draft history</i> berbasis <i>snapshot</i> , serta integrasi layanan AI eksternal melalui API untuk generasi narasi dan evaluasi skoring

BAB 3

DESKRIPSI SISTEM

Bab ini menguraikan rancangan platform penyusunan LED yang menjadi solusi atas permasalahan yang telah dirumuskan pada Bab 1. Pembahasan diawali dengan deskripsi solusi yang menjelaskan fitur-fitur utama platform secara argumentatif, dilanjutkan dengan desain sistem yang memaparkan arsitektur platform mulai dari pandangan menyeluruh hingga rincian rancangan pada lapisan *frontend* dan *backend*.

3.1 DESKRIPSI SOLUSI

Solusi yang ditawarkan dalam proyek akhir ini adalah sebuah platform web terintegrasi untuk penyusunan Laporan Evaluasi Diri (LED) akreditasi LAM-Teknik yang memadukan empat fitur utama untuk menjawab keempat permasalahan yang telah dirumuskan pada Bab 1.

1. Antarmuka penulisan terpadu berbasis *rich text editor* yang menyediakan lingkungan kerja tunggal untuk penyusunan narasi LED. Platform menyediakan editor berbasis TipTap yang memungkinkan pengguna merujuk data kuantitatif dari tabel LKPS secara langsung di dalam lingkungan kerja yang sama melalui tab referensi kontekstual, sehingga pengguna dapat memverifikasi angka yang dikutip dalam narasi tanpa perlu berpindah aplikasi dan risiko diskrepansi antara data riil dengan isi laporan dapat ditekan sejak proses penulisan berlangsung.
2. Lapisan orkestrasi *prompt* yang menjembatani permintaan pengguna dengan layanan *Retrieval-Augmented Generation* (RAG) melalui *template prompt* yang terstruktur. Setiap permintaan generasi narasi dari pengguna diolah menjadi *template* yang disesuaikan dengan konteks bab, sub-bab, dan sub-sub-bab LED yang sedang aktif, kemudian diteruskan ke layanan RAG untuk diproses bersama konteks rubrik LAM-Teknik yang relevan. Pendekatan ini memastikan narasi yang dihasilkan akurat, selaras dengan rubrik penilaian, dan minim risiko halusinasi data karena generasi berlangsung dalam konteks yang terkontrol.
3. Fitur skoring dan umpan balik secara *real-time* terhadap draf narasi yang sedang disusun. Platform menampilkan hasil evaluasi berdasarkan empat metrik RAGAS yang diterjemahkan ke dalam konteks penyusunan LED, yaitu akurasi bukti dan fakta, kesesuaian rubrik, kelengkapan panduan, dan efisiensi rujukan, disertai saran perbaikan yang dikategorikan berdasarkan tingkat prioritas. Untuk

mendukung transparansi proses evaluasi, platform juga menyediakan tab-tab kontekstual yang menampilkan referensi panduan LAM-Teknik, indikator rubrik, dan daftar bukti dukung yang diambil oleh layanan RAG sebagai konteks generasi dan penilaian narasi, sehingga pengguna dapat menelusuri dasar dari setiap skor yang diterima dan memperbaiki narasi secara terinformasi. Seluruh umpan balik ini disampaikan secara responsif melalui mekanisme *streaming* berbasis *Server-Sent Events* sehingga dapat ditampilkan secara bertahap tanpa perlu menunggu seluruh evaluasi selesai.

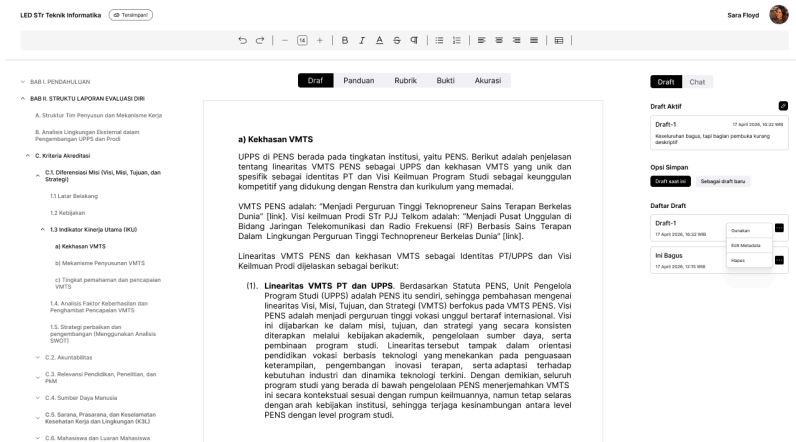
4. Sistem manajemen riwayat draf (*draft history*) yang adaptif terhadap iterasi penulisan berbantuan AI. Platform menyediakan dua lapisan persistensi, yaitu *auto-save* di latar belakang yang menyimpan draf aktif secara berkala untuk mencegah kehilangan tulisan, dan mekanisme *snapshot* eksplisit yang memungkinkan pengguna menyimpan versi draf yang dianggap baik beserta label penanda sebelum memicu regenerasi narasi baru. Pengguna dapat menjelajahi riwayat draf, membandingkan beberapa versi, serta memuat kembali versi yang diinginkan ke editor, sehingga progres penulisan tetap terjaga dan formulasi kalimat yang lebih baik tidak hilang selama proses iterasi.

3.2 DESAIN ANTARMUKA

Platform penyusunan LED dirancang dengan antarmuka berbasis satu halaman editor utama yang menerapkan tata letak tiga kolom adaptif, terdiri dari panel navigasi bab di sisi kiri, area konten bertab di tengah, dan panel kontekstual bertab di sisi kanan. Rincian masing-masing tampilan diuraikan pada sub-bab berikut.

3.2.1 Antarmuka Penulisan Narasi

Gambar 3.1 menampilkan tampilan platform pada kondisi tab *Draf* dan panel *Draft* aktif secara bersamaan, yang merupakan kondisi kerja paling umum selama proses penyusunan narasi.

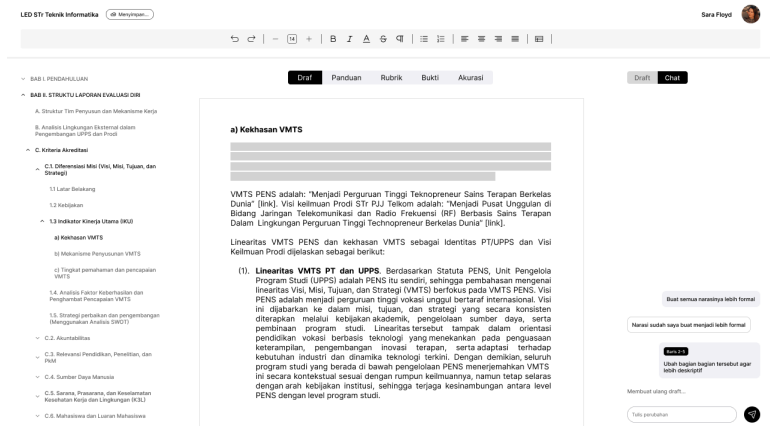


Gambar 3.1. Tampilan antarmuka penulisan narasi pada tab Draft dengan panel Draft aktif.

Area tengah menampilkan editor TipTap dengan *toolbar* pemformatan di bagian atas dan area narasi di bawahnya, sementara panel *Draft* di sisi kanan terbagi menjadi tiga bagian, yaitu *Draft Aktif* yang menampilkan versi draf yang sedang disunting beserta *timestamp* dan deskripsinya, *Opsi Simpan* yang menawarkan pilihan antara memperbarui draf saat ini atau menyimpan sebagai entri baru, serta *Daftar Draft* yang memuat seluruh *snapshot* tersimpan dengan opsi untuk menggunakan, menyunting metadata, atau menghapusnya. Indikator *Tersimpan* pada bagian atas halaman memberikan konfirmasi visual bahwa mekanisme *auto-save* sedang berjalan di latar belakang.

3.2.2 Interaksi Generasi Narasi AI

Gambar 3.2 menampilkan kondisi saat panel *Chat* diaktifkan di sisi kanan, sementara tab *Draft* di area tengah menunjukkan proses regenerasi narasi yang sedang berlangsung.

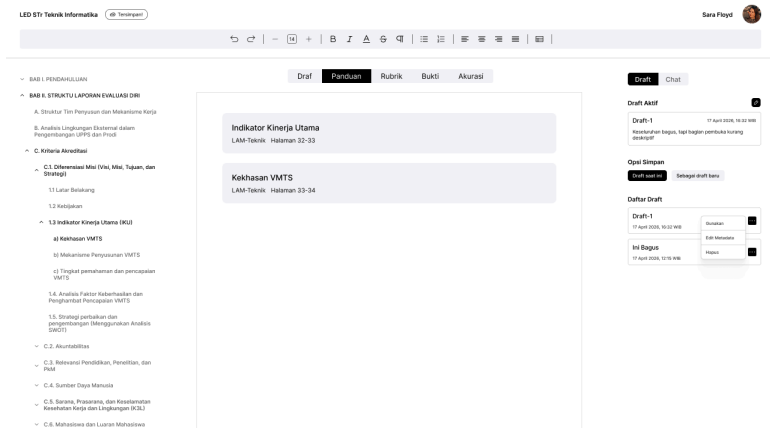


Gambar 3.2. Tampilan panel Chat untuk interaksi generasi narasi AI.

Alur interaksi yang ditampilkan diawali dari pengguna yang menyorot baris 2 hingga 5 pada editor, kemudian mengetikkan instruksi perbaikan melalui kolom *input* di bagian bawah panel *Chat*. Konteks seleksi teks tersebut secara otomatis dilampirkan pada pesan, ditandai dengan lencana *Baris 2-5* yang muncul pada entri percakapan terkait sehingga pengguna mengetahui bagian narasi mana yang menjadi target instruksi. Selama proses generasi ulang berlangsung, teks pada baris yang disorot digantikan sementara oleh blok abu-abu sebagai indikator *loading*, mencegah pengguna menyunting bagian tersebut sebelum hasil generasi disisipkan kembali ke editor. Indikator *Membuat ulang draft...* yang muncul di bawah riwayat percakapan memberikan konfirmasi bahwa sistem sedang memproses permintaan secara *streaming*.

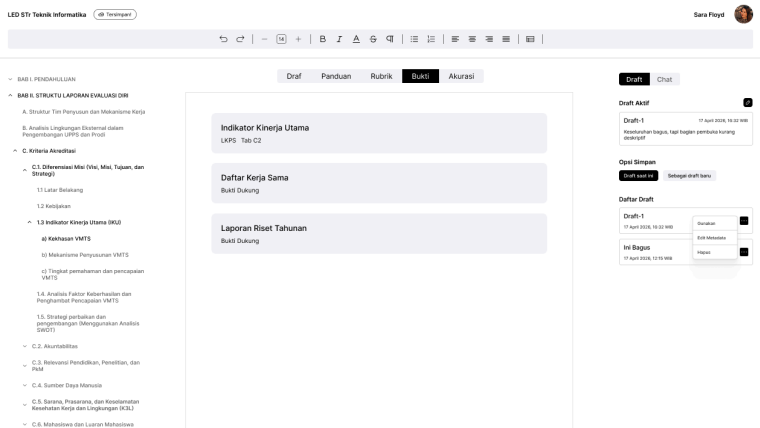
3.2.3 Tab Panduan, Bukti, dan Skor

Selain tab *Draft*, area konten tengah menyediakan tiga tab transparansi yang memperlihatkan konteks yang diambil oleh layanan RAG dalam proses generasi dan evaluasi narasi. Ketiga tab ini memungkinkan pengguna memverifikasi dasar dari narasi yang dihasilkan dan skor yang diterima, sehingga keluaran AI dapat ditelusuri asal-usul konteksnya secara langsung. Tab *Panduan* yang ditampilkan pada Gambar 3.3 menampilkan daftar referensi panduan LAM-Teknik yang diambil layanan RAG sebagai konteks untuk sub-bab yang sedang aktif, dilengkapi nomor halaman sumber sehingga pengguna dapat menelusuri dokumen aslinya.



Gambar 3.3. Tampilan tab Panduan

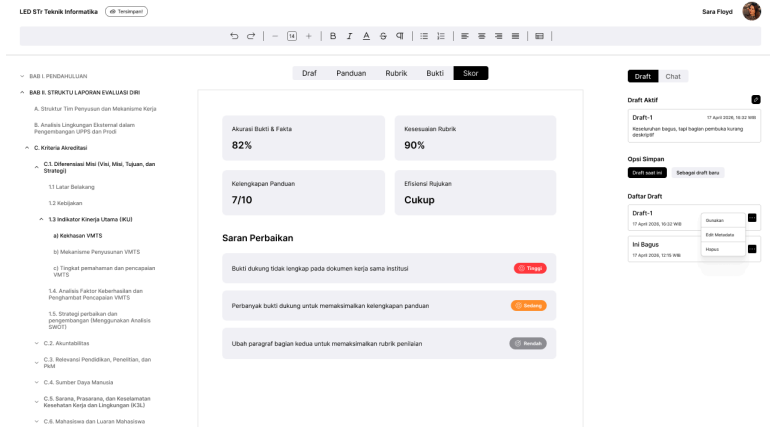
Tab *Bukti* pada Gambar 3.4 menampilkan daftar bukti dukung yang diidentifikasi layanan RAG sebagai dokumen yang diperlukan untuk mendukung narasi sub-bab terkait, lengkap dengan sumber referensinya.



Gambar 3.4. Tampilan tab Bukti

Tab *Skor* pada Gambar 3.5 menampilkan hasil evaluasi narasi berdasarkan empat metrik RAGAS yang diterjemahkan ke dalam konteks LED, yaitu akurasi bukti dan fakta (*faithfulness*), kesesuaian rubrik (*answer relevancy*), kelengkapan panduan (*context recall*), dan efisiensi rujukan

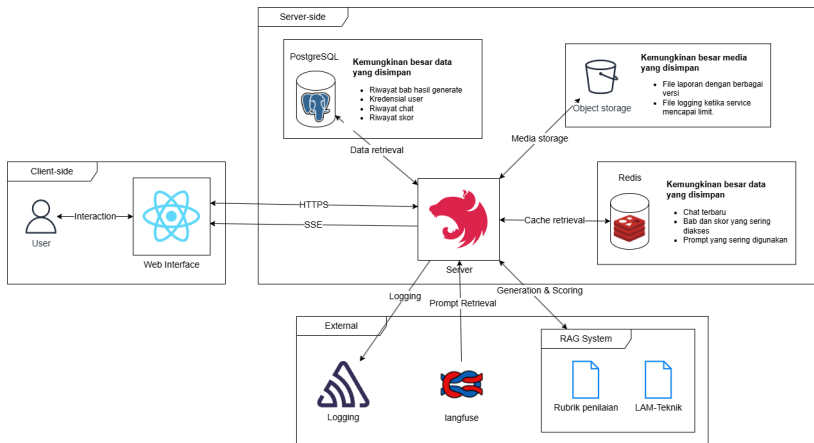
(*answer precision*), disertai daftar saran perbaikan yang dikategorikan berdasarkan tingkat prioritas tinggi, sedang, dan rendah.



Gambar 3.5. Tampilan tab Skor

3.3 DESAIN SISTEM

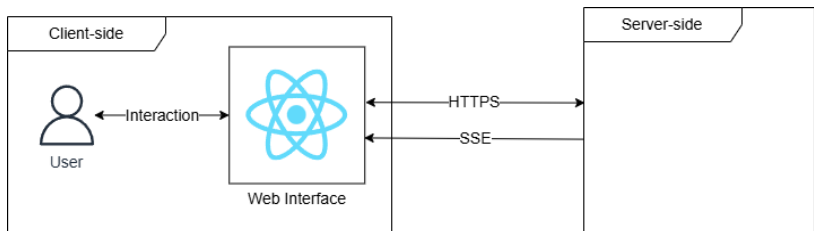
Secara keseluruhan, platform penyusunan LED dirancang dengan arsitektur yang terbagi menjadi tiga bagian utama, yaitu sisi klien (*client-side*), sisi peladen (*server-side*), dan layanan eksternal (*external*). Sisi klien merupakan antarmuka berbasis web yang menjadi titik interaksi pengguna dengan sistem, berkomunikasi dengan peladen melalui protokol HTTPS untuk pertukaran data umum dan *Server-Sent Events* (SSE) untuk penyaluran keluaran secara *streaming*. Sisi peladen berperan sebagai pusat pemrosesan yang menjembatani seluruh komponen, didukung oleh PostgreSQL sebagai basis data utama, Redis sebagai lapisan *cache*, serta *object storage* untuk penyimpanan berkas. Adapun layanan eksternal mencakup sistem RAG yang menyediakan kapabilitas generasi dan skoring berbasis rubrik penilaian, Langfuse sebagai *prompt management*, serta layanan *logging* untuk pencatatan aktivitas sistem. Gambar 3.6 menyajikan pandangan menyeluruh terhadap arsitektur sistem tersebut, yang selanjutnya diuraikan secara lebih rinci pada sub-bab berikutnya berdasarkan masing-masing bagian



Gambar 3.6. Arsitektur Keseluruhan

3.2.1 Sisi Klien

Sisi klien merupakan antarmuka berbasis web yang menjadi titik interaksi utama antara pengguna dan platform penyusunan LED. Antarmuka ini dibangun sebagai *Single Page Application* (SPA) menggunakan React, sehingga seluruh interaksi pengguna berlangsung dalam satu halaman yang dimuat sekali di awal dan diperbarui secara dinamis tanpa perlu memuat ulang halaman setiap kali terjadi perubahan. Pendekatan SPA dipilih karena platform menuntut interaksi yang intensif dan responsif, terutama pada aktivitas penyuntingan narasi, pemantauan skor, serta penjelajahan riwayat draf. Gambar 3.7 menyajikan potongan arsitektur pada sisi klien beserta hubungannya dengan peladen.



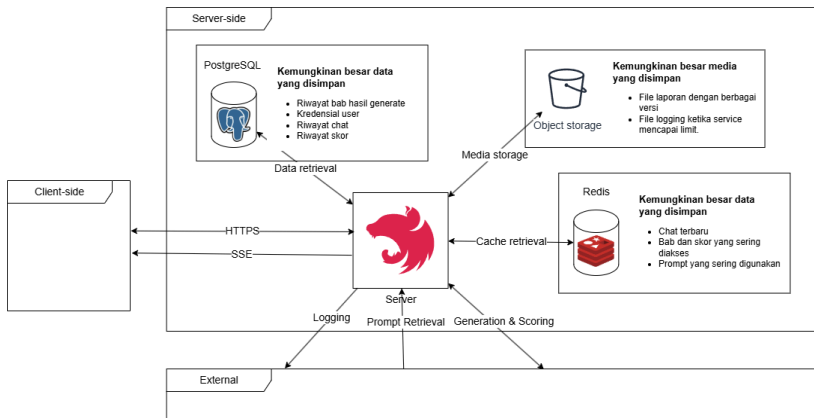
Gambar 3.7. Arsitektur Sisi Klien

Berdasarkan teknologi yang telah diuraikan pada Bab 2, antarmuka ini disusun dari beberapa komponen utama yang saling bekerja sama. Editor naskah dibangun menggunakan TipTap yang merepresentasikan konten dokumen dalam struktur JSON, sehingga narasi yang ditulis maupun yang dihasilkan oleh AI dapat disunting dan disimpan tanpa kehilangan informasi pemformatan. Pengelolaan *state* dibagi menjadi dua tanggung jawab, yaitu Zustand yang menangani *client state* seperti konteks bab yang sedang aktif dan status penyuntingan, serta TanStack Query yang menangani *server state* seperti data riwayat draf dan hasil skoring dengan mekanisme *cache* yang terpisah per bab. Pemisahan tersebut menjaga antarmuka tetap responsif sekaligus memastikan data dari peladen tetap konsisten ketika pengguna berpindah antar konteks bab.

Komunikasi antara sisi klien dan peladen berlangsung melalui dua kanal yang berbeda sesuai dengan karakteristik kebutuhannya. Kanal HTTPS digunakan untuk pertukaran data dengan pola *request-response* umum, seperti autentikasi pengguna, penyimpanan draf, serta pemuatan riwayat draf. Sementara itu, kanal *Server-Sent Events* (SSE) digunakan untuk menyalurkan keluaran yang bersifat bertahap dari peladen, yaitu hasil generasi narasi dan umpan balik skoring, sehingga konten dapat ditampilkan secara *streaming* kepada pengguna seiring diproduksi tanpa perlu menunggu seluruh proses selesai.

3.2.2 Sisi Peladen

Sisi peladen berperan sebagai pusat pemrosesan platform yang menjembatani permintaan dari sisi klien dengan seluruh komponen pendukung di belakangnya. Peladen dibangun menggunakan NestJS yang menangani berbagai tanggung jawab utama, mulai dari autentikasi pengguna, pengelolaan draf, penyaluran keluaran secara *streaming* ke sisi klien, hingga penyusunan *prompt* melalui lapisan orkestrasi sebelum permintaan diteruskan ke layanan eksternal. Gambar 3.8 menyajikan potongan arsitektur pada sisi peladen beserta komponen-komponen yang terhubung dengannya.



Gambar 3.8. Arsitektur Sisi Peladen

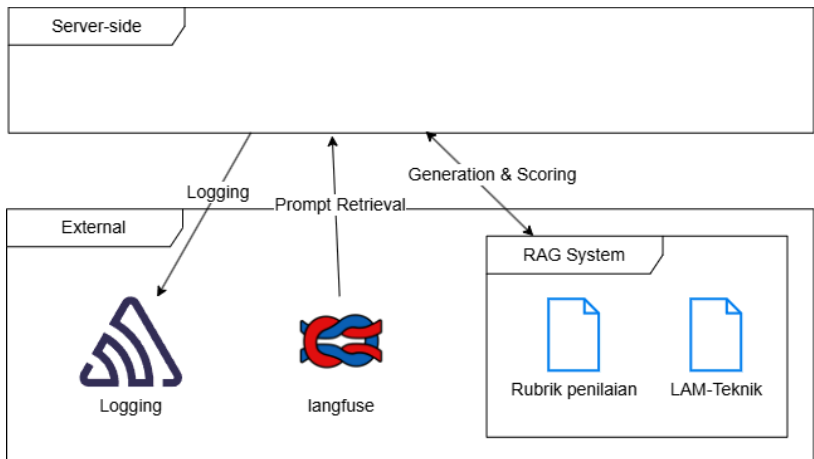
Dalam menjalankan tanggung jawabnya, peladen didukung oleh beberapa lapisan penyimpanan dengan peran yang berbeda. PostgreSQL berperan sebagai basis data utama yang menyimpan data persisten seperti riwayat bab hasil generasi, kredensial pengguna, riwayat percakapan, serta riwayat skor. Redis berperan sebagai lapisan *cache* yang menyimpan data yang sering diakses, seperti percakapan terbaru, bab dan skor yang kerap dibuka, serta *prompt* yang sering digunakan, sehingga mengurangi beban kueri berulang ke basis data utama dan mempercepat waktu respons. Selain kedua lapisan tersebut, peladen juga memanfaatkan *object storage* untuk menyimpan berkas berukuran besar seperti dokumen laporan dalam berbagai versi.

Selain berkomunikasi dengan lapisan penyimpanan, peladen juga terhubung dengan sejumlah layanan eksternal. Peladen mengakses sistem RAG untuk memperoleh kapabilitas generasi narasi dan skoring berbasis rubrik penilaian, mengambil *prompt* dari Langfuse yang berperan sebagai *prompt management*, serta mengirimkan catatan aktivitas sistem ke layanan *logging*. Rincian mengenai layanan-layanan eksternal tersebut diuraikan lebih lanjut pada sub-bab berikutnya.

3.2.3 Sisi Eksternal

Layanan eksternal mencakup sejumlah layanan di luar inti sistem yang diakses oleh peladen untuk mendukung fungsi-fungsi tertentu. Komponen utama pada bagian ini adalah sistem RAG yang menjadi sumber kapabilitas generasi narasi dan skoring berbasis rubrik penilaian. Peladen mengirimkan permintaan ke sistem RAG dan menerima keluaran berupa narasi maupun

hasil evaluasi skor, di mana basis pengetahuan sistem tersebut diisi dengan dokumen rubrik penilaian dan panduan akreditasi LAM-Teknik sebagai rujukan kontekstual. Sistem RAG itu sendiri dikembangkan oleh anggota tim lain dalam proyek akhir terpisah, sehingga platform yang dirancang penulis hanya berperan sebagai konsumen yang mengakses kapabilitas tersebut melalui kontrak API yang telah disepakati.



Gambar 3.9. Arsitektur Sisi Eksternal

Selain sistem RAG, peladen juga terhubung dengan dua layanan pendukung lainnya. Langfuse dimanfaatkan sebagai *prompt management*, yaitu tempat penyimpanan dan pengelolaan *prompt template* sehingga *prompt* dapat diperbarui tanpa perlu mengubah kode peladen. Sementara itu, layanan *logging* digunakan untuk pencatatan aktivitas dan kesalahan (*error*) sistem, sehingga jejak operasional platform dapat ditelusuri untuk keperluan pemantauan dan diagnosis ketika terjadi gangguan.

3.4 Use Case Diagram

Use case diagram menggambarkan interaksi antara pengguna dengan platform penyusunan LED dari sudut pandang fungsional, yaitu fungsi-fungsi apa saja yang dapat diakses pengguna tanpa memerinci bagaimana fungsi tersebut diimplementasikan. Sesuai dengan batasan penelitian yang berfokus pada penggunaan tunggal (*single-user*), diagram ini melibatkan satu aktor utama, yaitu Penyusun LED, yang merupakan

pengguna yang menyusun dokumen LED melalui platform. Gambar 3.10 menyajikan *use case diagram* platform penyusunan LED.



Gambar 3.10. Use Case Diagram

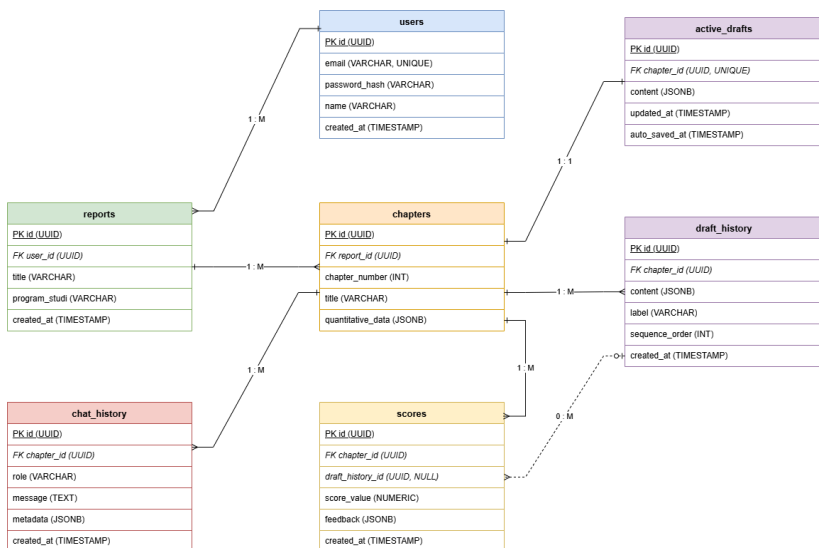
Aktor Penyusun LED dapat mengakses delapan fungsi utama pada platform. Pertama, fungsi Autentikasi yang mencakup proses Login dan Register sebagai pintu masuk pengguna ke dalam sistem. Kedua, fungsi Kelola Bab yang memungkinkan pengguna menavigasi antar bab LED dan menentukan bab yang sedang aktif disunting. Ketiga, fungsi Generate Narasi via AI yang menjadi inti pemanfaatan *Generative AI* pada platform, di mana pengguna meminta sistem menghasilkan narasi secara otomatis. Keempat, fungsi Edit Narasi Manual yang memungkinkan pengguna

menyunting langsung narasi di dalam editor. Kelima, fungsi Lihat Skor dan Umpan Balik secara *real-time* terhadap draf yang sedang disusun. Keenam, fungsi Lihat Riwayat Percakapan AI untuk menelusuri interaksi sebelumnya dengan sistem. Ketujuh, fungsi Kelola Riwayat Draft untuk mengatur berbagai versi draf. Kedelapan, fungsi Ekspor Dokumen LED ke dalam format Word atau PDF.

Beberapa fungsi utama memiliki keterkaitan dengan fungsi lain melalui relasi *include* dan *extend*. Fungsi Generate Narasi via AI memiliki relasi *include* dengan Regenerate Narasi dan Sisipkan Narasi AI ke Posisi Tertentu, yang menunjukkan bahwa kedua fungsi tersebut merupakan bagian dari alur generasi narasi, serta relasi *extend* dengan Instruksi Tambahan yang bersifat opsional ketika pengguna ingin menambahkan arahan khusus pada permintaan generasi. Adapun fungsi Kelola Riwayat Draft memiliki relasi *extend* dengan Simpan Draft, Bandingkan Versi Draft, Pulihkan Draft, dan Hapus Draft, di mana keempatnya merupakan tindakan opsional yang dapat dipilih pengguna saat mengelola riwayat draf. Fungsi Simpan Draft selanjutnya memiliki relasi *extend* dengan Beri Label Draft, sehingga pengguna dapat memberikan penanda pada draf yang disimpan.

3.5 Desain Database

Basis data platform penyusunan LED dirancang menggunakan PostgreSQL dengan struktur relasional yang terdiri dari tujuh tabel utama. Gambar 3.11 menyajikan rancangan skema basis data beserta relasi antar tabel. Secara hierarkis, struktur data mengikuti alur dari pengguna ke laporan, lalu ke bab, yang menjadi pusat keterhubungan bagi sebagian besar entitas lainnya. Tabel *users* menyimpan kredensial dan identitas pengguna, yang memiliki relasi satu-ke-banyak dengan tabel *reports*, sehingga seorang pengguna dapat mengelola lebih dari satu laporan LED. Setiap laporan pada tabel *reports* selanjutnya memiliki relasi satu-ke-banyak dengan tabel *chapters* yang merepresentasikan bab-bab dalam laporan tersebut.



Gambar 3.11. Desain Database

Tabel *chapters* berperan sebagai pusat keterhubungan yang menautkan sejumlah entitas pendukung. Setiap bab memiliki tepat satu draf aktif pada tabel *active_drafts* melalui relasi satu-ke-satu, serta dapat memiliki banyak catatan pada tabel *draft_history*, *scores*, dan *chat_history* melalui relasi satu-ke-banyak. Pemisahan antara tabel *active_drafts* dan *draft_history* mencerminkan mekanisme manajemen draf yang telah dibahas pada Bab 2, di mana draf aktif menjadi tempat penyuntingan dan auto-save berlangsung, sedangkan riwayat draf menyimpan snapshot yang disimpan pengguna secara eksplisit beserta label penandanya.

Konten naratif pada tabel *active_drafts* dan *draft history*, serta data kuantitatif pada tabel *chapters*, disimpan menggunakan tipe data JSONB. Sebagaimana diuraikan pada Bab 2, pemilihan JSONB memungkinkan struktur dokumen berformat kaya yang dihasilkan oleh editor disimpan secara utuh tanpa kehilangan informasi pemformatan, sekaligus tetap dapat dikueri secara terstruktur ketika diperlukan. Adapun tabel *scores* menyimpan hasil evaluasi skor beserta umpan baliknya, dengan kolom rujukan opsional ke tabel *draft_history* sehingga skor dapat dikaitkan dengan versi draf tertentu apabila diperlukan, sementara tabel *chat_history* menyimpan rekam jejak interaksi pengguna dengan layanan AI sepanjang proses penyusunan.

3.6 Metrik Evaluasi Sistem

Sebuah metrik yang digunakan evaluasi sistem sebagai standar keberhasilan yang terukur untuk memverifikasi bahwa platform yang dirancang mampu menjawab permasalahan yang telah dirumuskan pada Bab 1. Setiap metrik dirancang selaras dengan empat tujuan spesifik penelitian sehingga hasil eksperimen pada Bab 4 dapat dinilai secara objektif berdasarkan kriteria yang telah disepakati sebelum implementasi dilakukan. Metrik evaluasi dikelompokkan ke dalam empat kategori yang masing-masing berkorespondensi dengan fitur utama platform, yaitu metrik antarmuka penulisan terpadu, metrik *prompt orchestration* dan akurasi narasi, metrik skoring *real-time*, serta metrik manajemen riwayat draf.

3.5.1 Metrik Antarmuka Penulisan Terpadu

Metrik ini mengukur keberhasilan antarmuka penulisan dalam menyatukan pengelolaan data kuantitatif dan area penulisan narasi dalam satu lingkungan kerja tunggal sesuai tujuan pertama penelitian. Terdapat tiga metrik yang dievaluasi sebagaimana tercantum pada Tabel 3.1.

Tabel 3.1. Metrik Antarmuka Penulisan Terpadu

No	Metrik	Target	Metode Pengukuran
1	Konsistensi Data	100%	Manual audit perbandingan angka di tabel dengan angka yang dikutip pada narasi
2	Responsivitas Editor	Latensi pengetikan $\leq 100\text{ms}$, perpindahan antar bab ≤ 1 detik	Pengukuran waktu menggunakan browser performance tools
3	Keandalan Auto-save	Draf aktif tersimpan ≤ 5 detik setelah perubahan terakhir	Pengukuran interval waktu antara perubahan terakhir hingga konfirmasi penyimpanan dari basis data

Konsistensi data ditetapkan dengan target 100% karena dokumen LED bersifat formal dan menuntut *zero-tolerance* terhadap diskrepansi antara angka pada tabel dan narasi. Pengukurannya dilakukan dengan mengambil sejumlah sampel narasi hasil generasi maupun penulisan manual,

mengeksktraksi seluruh angka yang dikutip, lalu membandingkannya satu per satu dengan angka pada tabel kinerja yang menjadi sumbernya. Target responsivitas editor mengacu pada standar web performance yang menetapkan 100ms sebagai batas atas latensi interaksi agar pengguna tidak merasakan jeda yang mengganggu, sementara batas 1 detik untuk perpindahan bab ditetapkan agar konteks kerja pengguna tidak terputus. Keduanya diukur menggunakan browser performance tools pada skenario penggunaan normal. Keandalan *auto-save* ditargetkan tidak melebihi 5 detik untuk memastikan kehilangan data akibat gangguan koneksi atau crash aplikasi dapat diminimalkan, dan diukur dengan mencatat selisih waktu antara perubahan terakhir yang dilakukan pengguna dengan timestamp konfirmasi penyimpanan yang diterima dari basis data.

3.5.2 Metrik Prompt Orchestration dan Akurasi Narasi

Metrik ini mengukur keberhasilan lapisan *prompt orchestration* dalam menjembatani data pengguna dengan layanan RAG sesuai tujuan kedua penelitian. Terdapat tiga metrik yang dievaluasi sebagaimana tercantum pada Tabel 3.2.

Tabel 3.2. Metrik Prompt Orchestration dan Akurasi Narasi

No	Metrik	Target	Metode Pengukuran
1	Request-Response Conformance	100%	API validation testing terhadap OpenAPI schema yang disepakati bersama tim RAG
2	Latency Orchestration Layer	≤ 500ms (tidak termasuk processing time RAG)	Server-side logging: selisih timestamp request masuk hingga diteruskan ke RAG
3	Error Handling Coverage	100%	Fault injection testing: simulasi skenario RAG failure dan verifikasi logging, retry, serta notifikasi ke frontend

Request-Response Conformance ditetapkan dengan target 100% karena komunikasi yang tidak sesuai kontrak API dapat mengakibatkan kegagalan *request* atau *response* yang tidak dapat diproses oleh *frontend*, sehingga tidak ada toleransi terhadap ketidaksesuaian format.

Pengukurannya dilakukan dengan mengambil sampel seluruh *request* yang dikirim dari *orchestration layer* ke RAG beserta *response* yang diterima selama sesi testing, lalu memvalidasi keduanya terhadap OpenAPI *schema* yang telah didefinisikan bersama tim RAG untuk memastikan setiap field, tipe data, dan struktur payload sesuai kontrak yang disepakati. *Latency Orchestration Layer* ditargetkan tidak melebihi 500ms karena merupakan *threshold* yang *reasonable* untuk operasi *server-side* tanpa memperhitungkan processing time RAG yang berada di luar kendali penelitian ini. Pengukurannya dilakukan dengan *server-side logging*, yaitu mencatat *timestamp* pada saat request diterima oleh *orchestration layer* dan *timestamp* pada saat request tersebut diteruskan ke RAG, kemudian menghitung selisih keduanya pada *N requests* yang diambil selama sesi testing untuk mendapatkan nilai rata-rata dan nilai maksimum *latency*. *Error Handling Coverage* ditargetkan 100% untuk memastikan setiap kegagalan yang berasal dari RAG ditangani secara proper sehingga sistem tidak berada dalam state yang tidak konsisten dan pengguna selalu mendapat notifikasi yang informatif. Pengukurannya dilakukan dengan fault injection testing, yaitu mensimulasikan berbagai skenario kegagalan RAG seperti *timeout*, *error response*, dan *invalid payload*, kemudian memverifikasi bahwa *orchestration layer* merespons setiap skenario tersebut dengan melakukan *logging*, menjalankan *retry logic* jika diperlukan, dan mengirimkan pesan error yang informatif ke *frontend*.

3.5.3 Metrik Skoring Real-time

Metrik ini mengukur keberhasilan fitur skoring dalam menyampaikan umpan balik evaluasi secara responsif kepada pengguna melalui mekanisme streaming berbasis Server-Sent Events sesuai tujuan ketiga penelitian. Terdapat tiga metrik yang dievaluasi sebagaimana tercantum pada Tabel 3.3.

Tabel 3.3. Metrik Skoring Real-time

No	Metrik	Target	Metode Pengukuran
1	Latency Streaming Pertama	Event pertama diterima client ≤ 3 detik	Measure waktu dari request scoring hingga SSE event pertama diterima di frontend

No	Kategori	Metrik	Target
2	Konsistensi Skor	Variance skor $\leq$ 10% untuk narasi yang sama	Generate scoring untuk narasi identik sebanyak N kali, hitung variance skor yang dihasilkan
3	Keandalan Koneksi SSE	$\geq$ 99% sesi streaming selesai tanpa disconnect prematur	Monitor jumlah sesi streaming yang selesai normal versus yang terputus sebelum selesai selama sesi testing

Latency Streaming Pertama ditargetkan tidak melebihi 3 detik karena *threshold* tersebut merupakan batas toleransi pengguna terhadap waktu tunggu sebelum melihat respons pertama dari sistem, dan melampaui batas ini dapat menimbulkan kesan sistem tidak responsif. Pengukurannya dilakukan dengan mencatat *timestamp* pada saat *request scoring* dikirim dari frontend dan *timestamp* pada saat SSE *event* pertama diterima oleh *client*, kemudian menghitung selisih keduanya pada N sesi *scoring* selama pengujian untuk mendapatkan nilai rata-rata dan nilai maksimum *latency*. Konsistensi Skor ditargetkan dengan *variance* tidak melebihi 10% untuk memastikan bahwa pengguna mendapatkan umpan balik yang stabil dan dapat diandalkan sebagai indikator kualitas narasi. Pengukurannya dilakukan dengan mengirimkan narasi yang identik ke sistem *scoring* sebanyak N kali, mencatat skor yang dihasilkan pada setiap percobaan, lalu menghitung *variance* dari keseluruhan hasil tersebut. Keandalan Koneksi SSE ditargetkan minimal 99% sesi *streaming* selesai tanpa *disconnect* prematur untuk memastikan pengguna selalu menerima umpan balik secara lengkap tanpa terputus di tengah proses. Pengukurannya dilakukan dengan memonitor seluruh sesi *streaming* selama sesi *testing*, mencatat jumlah sesi yang selesai secara normal hingga *event* terakhir diterima dibandingkan dengan jumlah sesi yang terputus sebelum selesai, kemudian menghitung persentase keberhasilannya.

3.5.4 Metrik Manajemen Riwayat Draf

Metrik ini mengukur keberhasilan sistem manajemen riwayat draf dalam menjaga persistensi dan integritas seluruh iterasi penulisan yang dilakukan pengguna sesuai tujuan keempat penelitian. Terdapat empat metrik yang dievaluasi sebagaimana tercantum pada Tabel 3.4.

Tabel 3.4. Metrik Manajemen Riwayat Draf

No	Metrik	Target	Metode Pengukuran
1	Integritas Snapshot	100%	Simpan N snapshot, pulihkan masing-masing, bandingkan konten hasil pemulihan dengan konten original secara byte-by-byte
2	Latency Penyimpanan Snapshot	≤ 2 detik	Server-side logging: selisih timestamp antara aksi simpan pengguna hingga konfirmasi penyimpanan dari basis data
3	Latency Pemulihan Draf	≤ 1 detik	Measure waktu dari permintaan pemulihan hingga konten draf selesai dirender pada editor
4	Keandalan Auto-save	100% perubahan tersimpan ke draf aktif sebelum simulasi crash	Simulasi crash aplikasi pada berbagai interval pengeditan, verifikasi konten draf aktif di basis data

Integritas *Snapshot* ditetapkan dengan target 100% karena kehilangan atau perubahan konten pada snapshot yang tersimpan dapat mengakibatkan pengguna kehilangan formulasi narasi yang sebelumnya dianggap baik, sehingga tidak ada toleransi terhadap *data loss* pada mekanisme ini. Pengukurannya dilakukan dengan menyimpan N snapshot dari berbagai kondisi konten editor, memulihkan masing-masing snapshot ke editor, lalu membandingkan konten hasil pemulihan dengan konten original yang dicatat sebelum penyimpanan untuk memverifikasi tidak ada perbedaan yang terjadi. *Latency* Penyimpanan Snapshot ditargetkan tidak melebihi 2 detik untuk memastikan pengguna tidak mengalami jeda yang mengganggu alur kerja saat memutuskan untuk menyimpan versi draf tertentu. Pengukurannya dilakukan dengan *server-side logging*, yaitu mencatat *timestamp* pada saat aksi simpan diterima oleh peladen dan *timestamp* pada saat konfirmasi penyimpanan berhasil dikirim kembali ke *frontend*,

kemudian menghitung selisih keduanya pada N aksi penyimpanan selama sesi *testing*. *Latency* Pemulihan Draf ditargetkan tidak melebihi 1 detik karena pemulihan draf merupakan operasi yang dipicu secara eksplisit oleh pengguna dan hasilnya harus segera terlihat pada editor agar pengalaman pengguna tetap mulus. Pengukurannya dilakukan dengan mencatat *timestamp* pada saat permintaan pemulihan dikirim dari *frontend* dan *timestamp* pada saat konten draf selesai dirender pada editor, kemudian menghitung selisih keduanya pada N operasi pemulihan. Keandalan *Auto-save* ditargetkan 100% untuk memastikan tidak ada perubahan yang hilang akibat gangguan tak terduga seperti crash aplikasi atau koneksi terputus. Pengukurannya dilakukan dengan mensimulasikan crash aplikasi pada berbagai interval pengeditan, kemudian memverifikasi bahwa konten draf aktif yang tersimpan di basis data sesuai dengan perubahan terakhir yang dilakukan pengguna sebelum crash terjadi.

3.5.5 Ringkasan Metrik Evaluasi Sistem

Seluruh metrik evaluasi yang telah diuraikan pada sub-bab sebelumnya dirangkum pada Tabel 3.5 berikut sebagai acuan pelaksanaan pengujian pada Bab 4.

Tabel 3.5. Ringkasan Metrik Evaluasi Sistem

No	Kategori	Metrik	Target
1	Antarmuka Penulisan Terpadu	Konsistensi Data	100%
2	Antarmuka Penulisan Terpadu	Responsivitas Editor	Latensi pengetikan $\leq$ 100ms, perpindahan antar bab $\leq$ 1 detik
3	Antarmuka Penulisan Terpadu	Keandalan Auto-save	$\leq$ 5 detik setelah perubahan terakhir
4	Prompt Orchestration & Integrasi RAG	Request-Response Conformance	100%
5	Prompt Orchestration & Integrasi RAG	Latency Orchestration Layer	$\leq$ 500ms (excluding RAG processing time)

No	Kategori	Metrik	Target
6	Prompt Orchestration & Integrasi RAG	Error Handling Coverage	100%
7	Skoring Real-time	Latency Streaming Pertama	≤ 3 detik
8	Skoring Real-time	Konsistensi Skor	Variance $\leq 10\%$
9	Skoring Real-time	Keandalan Koneksi SSE	$\geq 99\%$ sesi selesai tanpa disconnect prematur
10	Manajemen Riwayat Draf	Integritas Snapshot	100%
11	Manajemen Riwayat Draf	Latency Penyimpanan Snapshot	≤ 2 detik
12	Manajemen Riwayat Draf	Latency Pemulihan Draf	≤ 1 detik
13	Manajemen Riwayat Draf	Keandalan Auto-save	100%

DAFTAR PUSTAKA

- [1] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D., *Language Models are Few-Shot Learners*, Advances in Neural Information Processing Systems (NeurIPS 2020), Vol. 33, Hal. 1877–1901, Curran Associates, Inc., 2020.
- [2] Coenen, A., Davis, L., Ippolito, D., Reif, E., & Yuan, A., *Wordcraft: a Human-AI Collaborative Editor for Story Writing*, arXiv preprint arXiv:2107.07430, <https://arxiv.org/abs/2107.07430>, 2021.
- [3] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H., *Retrieval-Augmented Generation for Large Language Models: A Survey*, arXiv preprint arXiv:2312.10997, <https://arxiv.org/abs/2312.10997>, 2024.
- [4] Hashemi, H., Eisner, J., Rosset, C., Van Durme, B., & Kedzie, C., *LLM-Rubric: A Multidimensional, Calibrated Approach to Automated Evaluation of Natural Language Texts*, Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Bangkok, Thailand, Hal. 13806–13834, Association for Computational Linguistics, 2024.
- [5] Hidayat, L. R., Pasek, G., Wijaya, S., & DwiYansaputra, R., *Optimalisasi Layanan Sistem Informasi Mahasiswa dengan Integrasi Telegram: Chatbot Retrieval-Augmented-Generation berbasis Large Language Model*, Jurnal Teknologi Informasi, Komputer dan Aplikasinya (JTika), Vol. 7, No. 1, Hal. 121–131, Universitas Mataram, 2025.
- [6] Komperla, V., Deenadhayalan, P., Ghuli, P., & Pattar, R., *React: A detailed survey*, Indonesian Journal of Electrical Engineering and Computer Science (IJECS), Vol. 26, No. 3, Hal. 1710–1717, Institute of Advanced Engineering and Science (IAES), 2022.
- [7] Kong, A., Zhao, S., Chen, H., Li, Q., Qin, Y., Sun, R., Zhou, X., Wang, E., & Dong, X., *Better Zero-Shot Reasoning with Role-Play Prompting*, Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Mexico City, Mexico, Hal. 4099–4113, Association for Computational Linguistics, 2024.
- [8] Menteri Pendidikan Tinggi, Sains, dan Teknologi Republik Indonesia, *Peraturan Menteri Pendidikan Tinggi, Sains, dan Teknologi Republik Indonesia Nomor 39 Tahun 2025 tentang Penjaminan Mutu*

- Pendidikan Tinggi*, Berita Negara Republik Indonesia Tahun 2025, Kementerian Pendidikan Tinggi, Sains, dan Teknologi, 2025.
- [9] Musumeci, E., Brienza, M., Suriani, V., Nardi, D., & Bloisi, D. D., *LLM-Based Multi-Agent Generation of Semi-structured Documents from Semantic Templates in the Public Administration Domain*, Artificial Intelligence in HCI (HCII 2024), Lecture Notes in Computer Science, Vol. 14736, Hal. 98–117, Springer, Cham, 2024.
 - [10] *NestJS Documentation*, Situs Resmi NestJS (docs.nestjs.com), <https://docs.nestjs.com>, Diakses tanggal 20 Mei 2026.
 - [11] *ProseMirror Guide (Documentation)*, Situs Resmi ProseMirror (prosemirror.net), <https://prosemirror.net/docs/guide>, Diakses tanggal 21 Mei 2026.
 - [12] Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S., & Chadha, A., *A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications*, arXiv preprint arXiv:2402.07927, <https://arxiv.org/abs/2402.07927>, 2024.
 - [13] Syaila, F., Azzahra, P., & Printo, C., *Tinjauan Literatur Sistematis: Analisis Implementasi Kecerdasan Buatan untuk Verifikasi Dokumen*, Jurnal Ilmiah Sistem Informasi (JUI SI), Vol. 4, No. 3, Hal. 417–430, LPPM Universitas Sains dan Teknologi Komputer (STEKOM), 2025.
 - [14] *TanStack Query Documentation*, Situs Resmi TanStack (tanstack.com), <https://tanstack.com/query/latest/docs/framework/react/overview>, Diakses tanggal 20 Mei 2026.
 - [15] *TipTap Editor Documentation*, Situs Resmi TipTap (tiptap.dev), <https://tiptap.dev/docs/editor>, Diakses tanggal 21 Mei 2026.
 - [16] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, Advances in Neural Information Processing Systems (NeurIPS 2022), Vol. 35, Hal. 24824–24837, Curran Associates, Inc., 2022.
 - [17] Zhang, Z., Du, F., Liu, X., Chen, P., Huang, O., Ye, R., Liut, M., & Aspuru-Guzik, A., *TreeWriter: AI-Assisted Hierarchical Planning and Writing for Long-Form Documents*, arXiv preprint arXiv:2601.12740, <https://arxiv.org/abs/2601.12740>, 2026.
 - [18] *Zustand Documentation*, Situs Resmi Zustand (zustand.docs.pmnd.rs), <https://zustand.docs.pmnd.rs/>, Diakses tanggal 20 Mei 2026.
 - [19] *Lanfuse Documentations*, Situs Resmi Lanfuse (langfuse.com), <https://langfuse.com/docs>, Diakses tanggal 11 Juni 2026.